



DBMS COURSE PROJECT

Design and Implementation of a Database Management System for

Canteen Food Ordering and Billing Management System

Project 07

Submitted by

Abhiuday Singh	25WU0101006
Aryaman Jain	25WU0101017
Harsh Jha	25WU0101046

Course: **Database Management Systems**

Faculty Mentor: **Dr. Kiran Mayee**

School of Technology, Woxsen University

Academic Year: 2025–2029

Abstract

The Canteen Food Ordering and Billing Management System is a database-driven web application that lets students order food from the campus canteen in advance and lets canteen staff run orders, stock and billing from one place. It addresses common canteen problems: long queues, orders written on paper, repeated entry of the same details, no visibility of order status, mistakes while preparing bills, and ingredients running out without warning.

The logical design is an Entity-Relationship model of ten entities — CUSTOMER, STAFF, CATEGORY, FOOD_ITEM, MENU_ITEM, INVENTORY, FOOD_ORDER, ORDER_ITEM, BILL and PAYMENT — mapped to a relational schema of eleven relations using the standard mapping rules, including an associative relation for the many-to-many “uses” relationship. The physical database, canteen_db, is implemented in MySQL 8 as ten InnoDB tables and one view. Primary keys, foreign keys with explicit delete rules, UNIQUE and CHECK constraints and ENUM domains enforce entity, referential and domain integrity inside the database, and the implemented schema is in Third Normal Form (3NF).

The application uses Node.js and Express on the server and plain HTML, CSS and JavaScript in the browser, connected to MySQL through the mysql2 driver with parameterized queries. Placing an order is a single ACID transaction that records the order and its lines at the current price, deducts ingredient stock using each item’s recipe, generates a bill with 5% GST and records the payment; if any step fails, nothing is saved. This report documents the design, the SQL used (DDL, DML, joins, aggregates, subqueries and a view) with outputs captured from the running database, screenshots of the working application, and 24 integrity and functional tests, all of which passed.

Keywords: *relational database, MySQL, ER model, normalization, 3NF, primary key, foreign key, transactions, canteen management, billing, SQL, CRUD, Node.js, Express.*

Table of Contents

Abstract.....	i
1. Introduction and Problem Statement.....	1
1.1 Introduction.....	1
1.2 Problem Statement.....	1
1.3 Intended Users.....	2
2. Objectives and Scope.....	3
2.1 Objectives.....	3
2.2 Scope.....	3
3. Software and Hardware Requirements.....	4
3.1 Software Requirements.....	4
3.2 Hardware Requirements.....	4
3.3 Functional Requirements.....	5
4. ER Diagram.....	6
4.1 Entities and Key Attributes.....	7
4.2 Relationships, Cardinality and Participation.....	8
4.3 Design Notes.....	8
5. Relational Schema and Normalization.....	9
5.1 Relational Schema.....	9
5.2 Functional Dependencies.....	11
5.3 Normalization to Third Normal Form.....	12
5.4 Result.....	12
5.5 From Logical Design to Physical Implementation.....	13
6. Data Dictionary.....	14
6.1 CUSTOMER.....	14
6.2 STAFF.....	14
6.3 CATEGORY.....	14
6.4 MENU_ITEM.....	15
6.5 FOOD_ITEM.....	15
6.6 INVENTORY.....	16
6.7 MENU_ITEM_INVENTORY.....	16
6.8 FOOD_ORDER.....	16
6.9 ORDER_ITEM.....	17
6.10 BILL.....	17
6.11 PAYMENT.....	17
7. SQL Commands Used (DDL and DML) with Sample Outputs.....	19
7.1 Physical Schema Overview.....	19
7.2 Data Definition Language (DDL).....	20
7.3 Data Manipulation Language (DML).....	25
7.4 Transactional Order Placement.....	28
7.5 Sample Data Overview.....	29

8. Queries with Outputs.....	30
8.1 Join query: customer order history.....	30
8.2 Aggregate query: revenue by category.....	30
8.3 Aggregate query: paid bills, revenue and average bill value.....	31
8.4 Popular food items.....	31
8.5 Active (not yet collected) orders.....	32
8.6 Ingredients at or below reorder level.....	32
8.7 Payment attempts by method and status.....	33
8.8 Recipe of a menu item (M:N join through MENU_ITEM_INGREDIENTS).....	33
8.9 Subquery: menu items priced above the average menu price.....	34
8.10 View: v_order_summary.....	34
9. UI Design and Screenshots.....	36
9.1 Design Approach.....	36
9.2 Screens.....	36
9.3 Insert, Update, View and Delete — Reflected in the Database.....	41
10. Implementation Details.....	48
10.1 Technology Stack and Architecture.....	48
10.2 Database Connectivity.....	48
10.3 Key Code Snippets.....	49
10.4 REST API.....	52
10.5 Order Processing Flow.....	53
10.6 Integrity and Security Measures.....	53
11. Testing.....	55
11.1 Testing Approach.....	55
11.2 Database Integrity Test Cases.....	55
11.3 Functional Test Cases.....	56
12. Conclusion and Future Enhancements.....	57
12.1 Conclusion.....	57
12.2 Future Enhancements.....	57
13. References.....	58
14. Contribution of Each Member.....	59
Appendix A: GitHub Repository.....	60
Appendix B: How to Run the Project.....	61

List of Figures

Figure 1: Entity–Relationship diagram of the Canteen Food Ordering and Billing Management System.....	6
Figure 2: Relational schema diagram: arrows run from each foreign key to the primary key it references	10
Figure 3: Physical database schema of canteen_db as implemented in database.sql.....	19
Figure 4: Login screen with sample accounts for each role.....	36
Figure 5: Student menu: categories, search, prices, preparation time and ratings.....	37
Figure 6: Tray and checkout: line totals, 5% GST, pickup point and payment method; sold-out items cannot be added.....	37
Figure 7: My orders: pickup token, lane, status progress and payment state after placing an order.....	38
Figure 8: Bill generated for the order, with GST, total and UPI transaction reference.....	38
Figure 9: Staff / admin dashboard: revenue, active orders, low stock, best sellers and sales by category	39
Figure 10: Order management: active orders with payment state and the next status action.....	39
Figure 11: Menu management (admin): items with availability toggle, edit and delete; categories below	40
Figure 12: Inventory: stock and reorder levels, with ingredients below the reorder level flagged.....	40
Figure 13: Billing: every bill with GST, discount, payment state, reference and cash collection.....	41
Figure 14: Registration form: a new student account is created.....	42
Figure 15: Staff moved the order to “Preparing”; the confirmation message is shown.....	44
Figure 16: Add menu item form filled in by the administrator.....	45
Figure 17: The new item appears in the menu table (highlighted) with the “Item added” message.....	46
Figure 18: Deleting Chicken Biryani is refused because the item appears in past orders.....	46
Figure 19: The test item is deleted and no longer listed.....	47
Figure 20: System architecture.....	48
Figure 21: Order placement transaction: the six steps and how it ends.....	53
Figure 22: Order status lifecycle implemented in the application.....	53

List of Tables

Table 1: Intended users and their roles.....	2
Table 2: Project scope.....	3
Table 3: Software requirements.....	4
Table 4: Hardware requirements.....	4
Table 5: Functional requirements.....	5
Table 6: Entities and key attributes.....	7
Table 7: Relationships, cardinality and participation.....	8
Table 8: Foreign keys of the relational schema.....	11
Table 9: Functional dependencies.....	11
Table 10: Normalization result.....	12
Table 11: Mapping from the logical schema to the physical tables.....	13
Table 12: Data dictionary: CUSTOMER.....	14
Table 13: Data dictionary: STAFF.....	14
Table 14: Data dictionary: CATEGORY.....	15
Table 15: Data dictionary: MENU_ITEM.....	15
Table 16: Data dictionary: FOOD_ITEM.....	15
Table 17: Data dictionary: INVENTORY.....	16
Table 18: Data dictionary: MENU_ITEM_INVENTORY.....	16
Table 19: Data dictionary: FOOD_ORDER.....	16
Table 20: Data dictionary: ORDER_ITEM.....	17
Table 21: Data dictionary: BILL.....	17
Table 22: Data dictionary: PAYMENT.....	18
Table 23: Rows loaded by database.sql.....	29
Table 24: Output of Query 1.....	30
Table 25: Output of Query 2.....	31
Table 26: Output of Query 3.....	31
Table 27: Output of Query 4.....	32
Table 28: Output of Query 5.....	32
Table 29: Output of Query 6.....	33
Table 30: Output of Query 7.....	33
Table 31: Output of Query 8.....	34
Table 32: Output of Query 9.....	34
Table 33: Output of Query 10.....	35
Table 34: Technology stack.....	48
Table 35: REST API endpoints.....	52
Table 36: Database integrity test cases.....	55
Table 37: Functional test cases.....	56
Table 38: Contribution of each member.....	59
Table 39: Repository details.....	60
Table 40: Sample login accounts.....	61

1. Introduction and Problem Statement

1.1 Introduction

A campus canteen handles a continuous stream of small but related transactions. A customer chooses one or more items, the order is recorded, the kitchen prepares the items using ingredients from stock, the order status changes, and a bill is settled through a payment method. When these activities are managed on paper or in disconnected records, the same information is entered repeatedly, stock runs out without warning, and it becomes hard to answer simple questions such as which items are available, how many orders a customer placed, or how much revenue a category generated.

A database management system provides a single structured source of truth for these activities. This project designs that database from an Entity-Relationship model, maps it to a normalized relational schema, implements it in MySQL 8 with integrity constraints, and builds a working web application on top of it. Students order from their browser and receive a pickup token; canteen staff move orders through the kitchen, collect cash, maintain stock and view bills; administrators also maintain the menu. Every screen reads from and writes to the same MySQL database.

1.2 Problem Statement

Design and implement a database management system for canteen food ordering and billing that maintains user and menu information, records orders and their individual items at the price charged, tracks order status, keeps ingredient stock in step with orders, generates bills, records payment attempts, and supports operational reporting through SQL queries.

The system must ensure that:

- each order belongs to an existing registered user, and each menu item to an existing category;
- each order line refers to a valid order and a valid menu item and stores the price at the time of ordering;
- an order cannot contain the same menu item twice;
- quantities, prices, stock levels and bill amounts cannot be negative, and a bill total always equals subtotal + tax – discount;
- every bill belongs to exactly one order, and every payment to an existing bill;
- ingredient stock is reduced when an order is placed and restored when it is cancelled, and an order is refused if stock is insufficient;
- only canteen staff and administrators can manage orders, stock and billing, and only administrators can change the menu.

1.3 Intended Users

Table 1: Intended users and their roles

User	Role in the database	What they do in the system
Student / Customer	STUDENT	Registers, browses the menu, places orders and pays (UPI, campus wallet or cash), tracks the order token, views and prints the bill, cancels an order before preparation starts.
Faculty / other campus users	STUDENT	Order through the same customer workflow as students.
Canteen Staff	STAFF	Views the dashboard, moves orders through the kitchen, collects cash payments, updates stock and reorder levels, views all bills.
Administrator	ADMIN	Everything staff can do, plus adding, editing and deleting menu items and categories and marking items sold out.

2. Objectives and Scope

2.1 Objectives

- Model canteen ordering, kitchen stock and billing as a clear Entity–Relationship design.
- Map the ER model to a relational schema and normalize it to Third Normal Form (3NF).
- Implement the schema in MySQL 8 with primary keys, foreign keys with explicit delete rules, UNIQUE, CHECK and ENUM constraints.
- Record every order as one ACID transaction covering the order, its lines, stock deduction, bill and payment.
- Keep a traceable order → bill → payment chain, including failed and pending payment attempts.
- Provide a web interface for inserting, viewing, updating and deleting records, with role-based access.
- Answer operational questions with JOIN, aggregate, subquery and view-based SQL queries.

2.2 Scope

Table 2: Project scope

In scope	Out of scope
Student registration and login (bcrypt-hashed passwords)	Raw-material procurement and supplier management
Category and menu management, sold-out toggle	Payroll and employee scheduling
Order placement, tracking, status flow and cancellation	External food-delivery logistics
Recipe-based ingredient stock and low-stock alerts	Full accounting / GST filing integration
Bill generation (5% GST, discount) and payment recording	Real payment-gateway processing (UPI / wallet are simulated)
Dashboard reports and SQL queries	Native mobile application

3. Software and Hardware Requirements

3.1 Software Requirements

Table 3: Software requirements

Component	Used in the project	Purpose
Database server	MySQL 8.0 (InnoDB, utf8mb4)	Stores the data and enforces keys, constraints and transactions.
Runtime	Node.js 18 or later	Runs the application server.
Web framework	Express 5	REST API routes and static file serving (<code>server.js</code>).
Database driver	mysql2 (promise API)	Connection pool and parameterized queries between Node.js and MySQL.
Password hashing	bcryptjs	Hashes passwords at registration and verifies them at login.
Frontend	HTML5, CSS3, JavaScript (no framework)	Menu, tray, orders, bill, dashboard, menu, inventory and billing screens.
Operating system	Windows / Linux / macOS	Runs MySQL, Node.js and the browser.
Browser	Chrome / Edge / Firefox	Runs the web interface.
Development tools	VS Code, MySQL Workbench / CLI, Git, GitHub	Coding, database inspection and version control.

3.2 Hardware Requirements

Table 4: Hardware requirements

Component	Minimum	Recommended
Processor	Dual-core 64-bit CPU	Quad-core or better
RAM	4 GB	8 GB or more
Storage	1 GB free	2 GB or more
Display	1280 × 720	1440 × 900 or larger
Network	Not required for a local setup	Stable network for a shared / hosted demo

3.3 Functional Requirements

Table 5: Functional requirements

ID	Requirement
FR-01	Students can register; all users log in with email and password; access depends on role (STUDENT, STAFF, ADMIN).
FR-02	Administrators can add, edit and delete categories and menu items; staff and administrators can mark items sold out.
FR-03	Only available items can be ordered; quantity per item is 1–20.
FR-04	A student can place an order with one or more items, choose a pickup point and a payment method.
FR-05	Order lines store quantity and the unit price at the time of ordering.
FR-06	Placing an order deducts ingredient stock using each item's recipe; the order is refused if stock is insufficient.
FR-07	Every order gets one bill with subtotal, 5% GST, discount and total.
FR-08	Each bill can have several payment attempts (PENDING, SUCCESS, FAILED); staff can record cash payments.
FR-09	Order status moves PENDING → CONFIRMED → PREPARING → READY → COMPLETED, or to CANCELLED; cancelling restores stock.
FR-10	Staff can view and edit stock and reorder levels; items at or below the reorder level are flagged.
FR-11	The dashboard shows revenue, order counts, best sellers, sales by category and payment-method totals.
FR-12	Every insert, update and delete made in the UI is stored in MySQL and visible through SELECT queries.

4. ER Diagram

The conceptual design is expressed as an Entity-Relationship diagram with ten entities and nine relationships. Main entities (green) hold the people and items the canteen deals with, transaction / associative entities (blue) record what happens, master / lookup entities (yellow) classify and describe items, and diamonds show the relationships with their cardinalities.

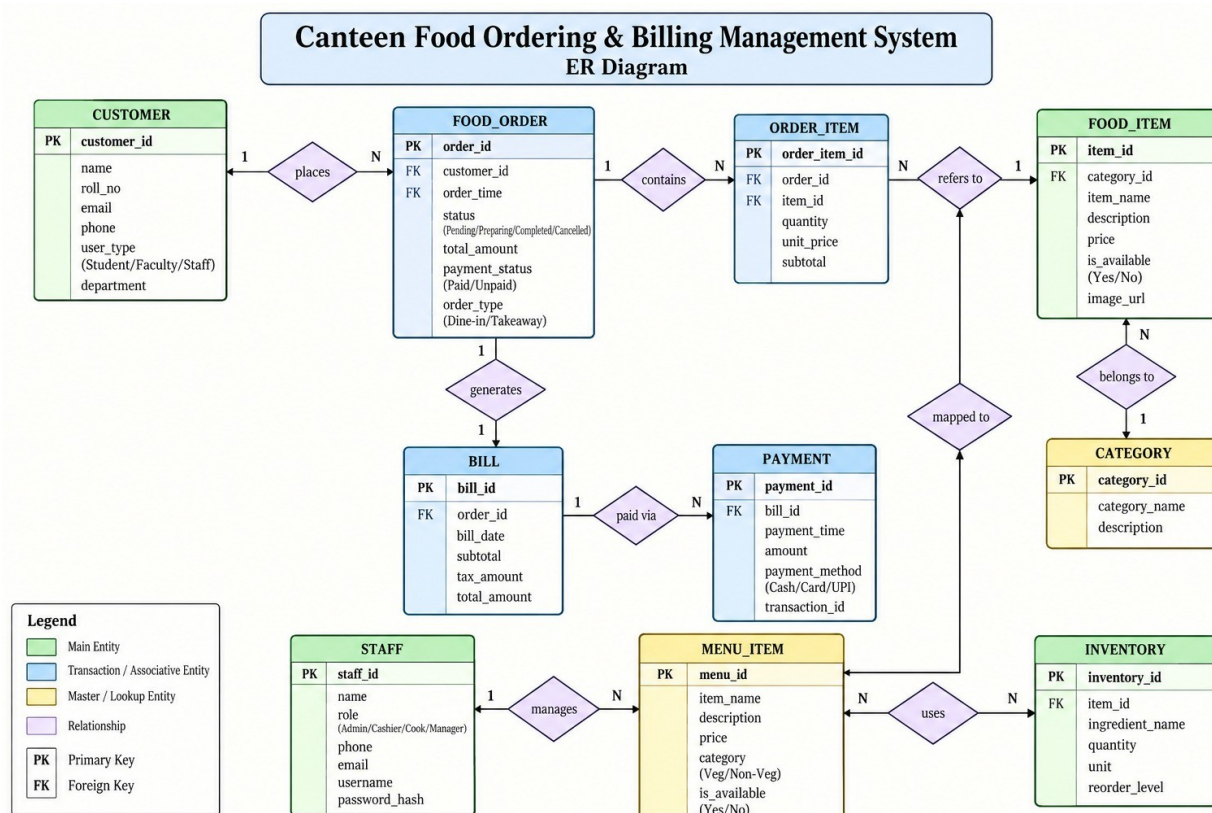


Figure 1: Entity-Relationship diagram of the Canteen Food Ordering and Billing Management System

4.1 Entities and Key Attributes

Table 6: Entities and key attributes

Entity	Type	Primary key	Attributes
CUSTOMER	Main	customer_id	name, roll_no, email, phone, user_type (Student / Faculty / Staff), department
STAFF	Main	staff_id	name, role (Admin / Cashier / Cook / Manager), phone, email, username, password_hash
CATEGORY	Master / lookup	category_id	category_name, description
FOOD_ITEM	Main	item_id	category_id (FK), item_name, description, price, is_available, image_url
MENU_ITEM	Master / lookup	menu_id	item_name, description, price, category (Veg / Non-Veg), is_available
INVENTORY	Main	inventory_id	item_id (FK), ingredient_name, quantity, unit, reorder_level
FOOD_ORDER	Transaction	order_id	customer_id (FK), order_time, status (Pending / Preparing / Completed / Cancelled), total_amount, payment_status (Paid / Unpaid), order_type (Dine-in / Takeaway)
ORDER_ITEM	Associative	order_item_id	order_id (FK), item_id (FK), quantity, unit_price, subtotal
BILL	Transaction	bill_id	order_id (FK), bill_date, subtotal, tax_amount, total_amount
PAYMENT	Transaction	payment_id	bill_id (FK), payment_time, amount, payment_method (Cash / Card / UPI), transaction_id

4.2 Relationships, Cardinality and Participation

Table 7: Relationships, cardinality and participation

Relationship	Entities	Cardinality	Participation / meaning
places	CUSTOMER – FOOD_ORDER	1 : N	Every order is placed by exactly one customer; a customer may place many orders (or none yet).
contains	FOOD_ORDER – ORDER_ITEM	1 : N	Every order contains at least one order line; each line belongs to one order.
refers to	ORDER_ITEM – FOOD_ITEM	N : 1	Each order line refers to one food item; a food item may appear in many orders.
belongs to	FOOD_ITEM – CATEGORY	N : 1	Every food item belongs to one category; a category groups many items.
generates	FOOD_ORDER – BILL	1 : 1	Each order generates one bill; each bill is for exactly one order.
paid via	BILL – PAYMENT	1 : N	A bill can have several payment attempts, e.g. a failed UPI attempt followed by cash.
manages	STAFF – MENU_ITEM	1 : N	A staff member manages many menu items; each menu item is managed by one staff member.
uses	MENU_ITEM – INVENTORY	M : N	A menu item uses many stock items and a stock item is used by many menu items.
mapped to	MENU_ITEM – FOOD_ITEM	1 : N	Each food item sold to customers is mapped to the kitchen menu item that produces it.

ORDER_ITEM is the associative entity that resolves the many-to-many relationship between orders and food items, while capturing the quantity and the price charged. The combination (order_id, item_id) is unique, so the same food item cannot appear as two separate lines in one order.

4.3 Design Notes

- **Derived attributes.** ORDER_ITEM.subtotal (quantity × unit_price), BILL.total_amount (subtotal + tax_amount), FOOD_ORDER.total_amount and FOOD_ORDER.payment_status can be calculated from other data. They are kept in the conceptual model for readability and handled explicitly during normalization (Section 5.4).
- **Relationship attributes.** The M : N “uses” relationship becomes its own relation in the schema; the quantity of each stock item used per serving belongs to that relation.
- **“mapped to”.** In the diagram this relationship is drawn towards the “refers to” relationship. During mapping it is treated as a relationship between MENU_ITEM and FOOD_ITEM (1 : N), which is the only reading that can be represented with a foreign key.
- **order_time** is a descriptive attribute of FOOD_ORDER (the time the order was placed), not a foreign key.

5. Relational Schema and Normalization

5.1 Relational Schema

The ER model is converted to relations with the standard mapping rules: each entity becomes a relation with its key as the primary key; for each 1 : N relationship the primary key of the “1” side becomes a foreign key on the “N” side; the 1 : 1 relationship becomes a foreign key with a UNIQUE constraint; and the M : N relationship becomes a new relation whose primary key combines both foreign keys. Primary keys are shown **bold and underlined**, foreign keys in *italics*, and attributes added by the mapping rules are marked with an asterisk (*).

CUSTOMER (**customer_id**, name, roll_no, email, phone, user_type, department)

STAFF (**staff_id**, name, role, phone, email, username, password_hash)

CATEGORY (**category_id**, category_name, description)

MENU_ITEM (**menu_id**, *staff_id**, item_name, description, price, category, is_available)

FOOD_ITEM (**item_id**, *category_id*, *menu_id**, item_name, description, price, is_available, image_url)

INVENTORY (**inventory_id**, *item_id*, ingredient_name, quantity, unit, reorder_level)

MENU_ITEM_INVENTORY* (*menu_id*, *inventory_id*)

FOOD_ORDER (**order_id**, *customer_id*, order_time, status, total_amount, payment_status, order_type)

ORDER_ITEM (**order_item_id**, *order_id*, *item_id*, quantity, unit_price, subtotal)

BILL (**bill_id**, *order_id*, bill_date, subtotal, tax_amount, total_amount)

PAYMENT (**payment_id**, *bill_id*, payment_time, amount, payment_method, transaction_id)

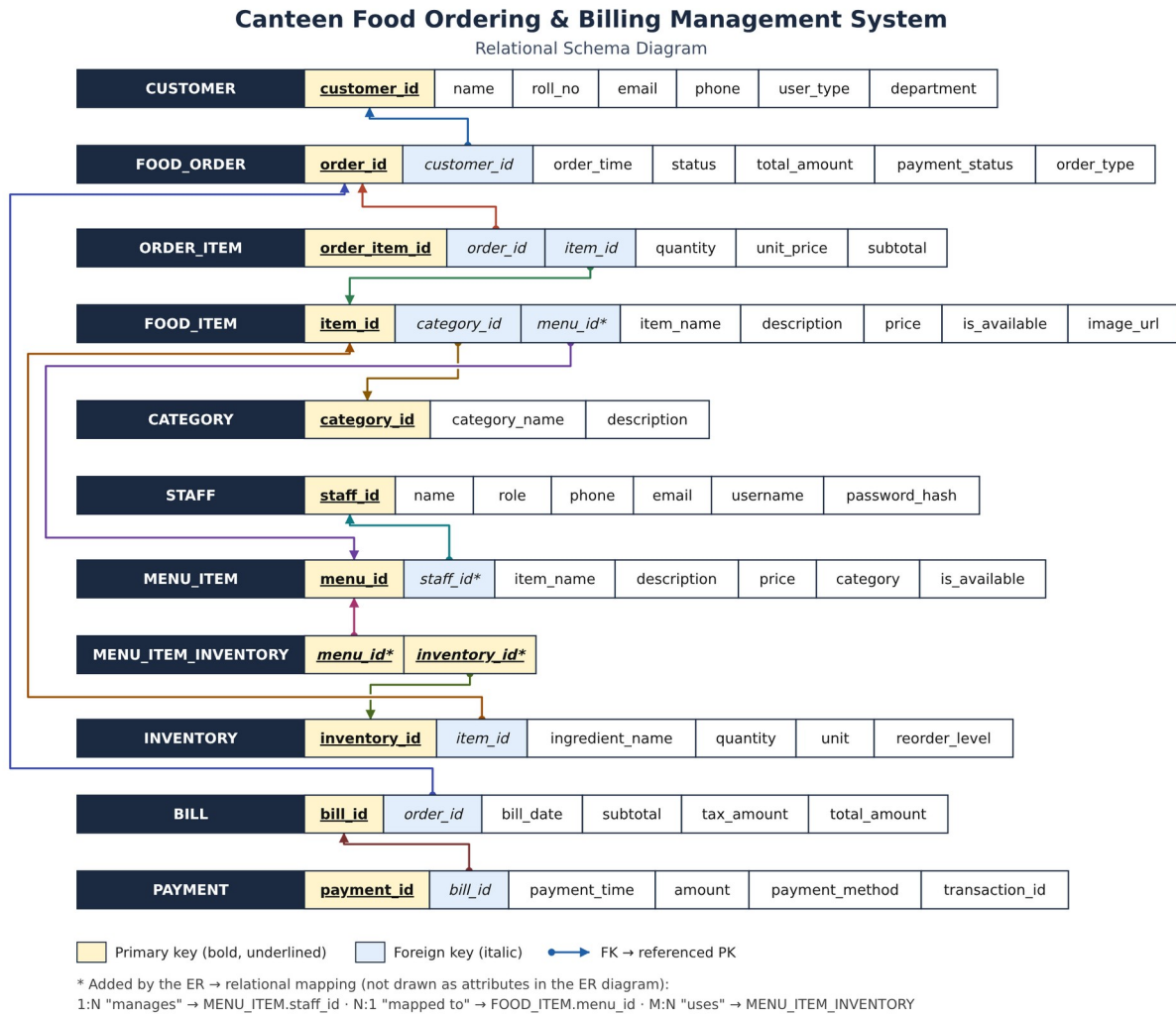


Figure 2: Relational schema diagram: arrows run from each foreign key to the primary key it references

Table 8: Foreign keys of the relational schema

Foreign key	References	Implements relationship
FOOD_ORDER.customer_id	CUSTOMER.customer_id	places (1 : N)
ORDER_ITEM.order_id	FOOD_ORDER.order_id	contains (1 : N)
ORDER_ITEM.item_id	FOOD_ITEM.item_id	refers to (N : 1)
FOOD_ITEM.category_id	CATEGORY.category_id	belongs to (N : 1)
FOOD_ITEM.menu_id	MENU_ITEM.menu_id	mapped to (1 : N)
MENU_ITEM.staff_id	STAFF.staff_id	manages (1 : N)
MENU_ITEM_INVENTORY.menu_id	MENU_ITEM.menu_id	uses (M : N)
MENU_ITEM_INVENTORY.inventory_id	INVENTORY.inventory_id	uses (M : N)
INVENTORY.item_id	FOOD_ITEM.item_id	stock item linked to a food item
BILL.order_id (UNIQUE)	FOOD_ORDER.order_id	generates (1 : 1)
PAYMENT.bill_id	BILL.bill_id	paid via (1 : N)

5.2 Functional Dependencies

Table 9: Functional dependencies

Relation	Functional dependencies
CUSTOMER	customer_id → name, roll_no, email, phone, user_type, department; email → customer_id; roll_no → customer_id
STAFF	staff_id → name, role, phone, email, username, password_hash; email → staff_id; username → staff_id
CATEGORY	category_id → category_name, description; category_name → category_id
MENU_ITEM	menu_id → staff_id, item_name, description, price, category, is_available; item_name → menu_id
FOOD_ITEM	item_id → category_id, menu_id, item_name, description, price, is_available, image_url; item_name → item_id
INVENTORY	inventory_id → item_id, ingredient_name, quantity, unit, reorder_level; ingredient_name → inventory_id
MENU_ITEM_INVENTORY	{menu_id, inventory_id} is the whole key (all-key relation, no non-trivial dependencies)
FOOD_ORDER	order_id → customer_id, order_time, status, total_amount, payment_status, order_type
ORDER_ITEM	order_item_id → order_id, item_id, quantity, unit_price, subtotal; {order_id, item_id} → order_item_id; {quantity, unit_price} → subtotal
BILL	bill_id → order_id, bill_date, subtotal, tax_amount, total_amount; order_id → bill_id; {subtotal, tax_amount} → total_amount
PAYMENT	payment_id → bill_id, payment_time, amount, payment_method, transaction_id; transaction_id → payment_id

5.3 Normalization to Third Normal Form

First Normal Form (1NF)

A flat canteen record might store several ordered items in one field, such as “Chicken Biryani ×1, Masala Chai ×2”, or several ingredients in one column of a menu record. The design removes these repeating groups: each item in an order is a separate ORDER_ITEM row, and each stock item used by a dish is a separate MENU_ITEM_INVENTORY row. Every attribute holds a single atomic value.

Second Normal Form (2NF)

2NF requires that no non-key attribute depends on only part of a composite key. The only composite primary key is {menu_id, inventory_id} in MENU_ITEM_INVENTORY, which has no non-key attributes, and every other relation has a single-attribute primary key. Attributes describing an order stay in FOOD_ORDER and attributes describing an item stay in FOOD_ITEM, so there are no partial dependencies.

Third Normal Form (3NF)

3NF requires that no non-key attribute depends on another non-key attribute. Customer, category, staff and stock details are stored once in their own relations and referenced by key instead of being repeated in every order row. Two stored values are exceptions because they are calculated: ORDER_ITEM.subtotal depends on {quantity, unit_price}, and BILL.total_amount depends on {subtotal, tax_amount}. These transitive dependencies are resolved in the implementation as described below.

5.4 Result

Table 10: Normalization result

Relation	1NF	2NF	3NF	Reason
CUSTOMER	Yes	Yes	Yes	All attributes depend only on customer_id (email and roll_no are candidate keys).
STAFF	Yes	Yes	Yes	All attributes depend only on staff_id.
CATEGORY	Yes	Yes	Yes	Category name is stored once.
MENU_ITEM	Yes	Yes	Yes	Staff details are referenced through staff_id, not copied.
FOOD_ITEM	Yes	Yes	Yes	Category and menu-item details are referenced through foreign keys.
INVENTORY	Yes	Yes	Yes	Stock data depends only on inventory_id.
MENU_ITEM_INVENTORY	Yes	Yes	Yes	All-key relation.
FOOD_ORDER	Yes	Yes	Yes*	total_amount and payment_status repeat data held in BILL and PAYMENT.
ORDER_ITEM	Yes	Yes	Yes*	subtotal is derived from quantity × unit_price.
BILL	Yes	Yes	Yes*	total_amount is derived from subtotal + tax_amount.
PAYMENT	Yes	Yes	Yes	Payment data depends only on payment_id.

* In the implemented database (Chapter 7) these derived values are handled so that the stored schema is in 3NF: the order-line total is **not stored** and is calculated as $\text{quantity} \times \text{unit_price}$ in queries and in the view `v_order_summary`; the order's amount and payment status are **not stored on the order** but read from the bill and its payments; and the bill keeps `total_amount` as a financial snapshot that is guarded by the constraint `CHECK (total_amount = subtotal + tax - discount)`, so it can never disagree with its parts. `unit_price` is deliberately stored, because it records the price at the time of ordering and cannot be derived once the menu price changes.

5.5 From Logical Design to Physical Implementation

The physical database in the project repository (`database.sql`) implements this design with some practical consolidations. Customers and staff share one `users` table with a role column, because both log in the same way; `FOOD_ITEM` and `MENU_ITEM` are combined into `menu_items`; and stock is split into `ingredients` (what an ingredient is) and `inventory` (how much is in stock), with `menu_item_ingredients` holding each recipe.

Table 11: Mapping from the logical schema to the physical tables

Logical relation	Physical table(s)	Notes
CUSTOMER, STAFF	<code>users</code>	One table; role = STUDENT, STAFF or ADMIN; bcrypt password_hash for every user.
CATEGORY	<code>categories</code>	Same structure.
FOOD_ITEM, MENU_ITEM	<code>menu_items</code>	One table with price, availability, preparation_time, rating and image_url.
INVENTORY	<code>ingredients</code> , <code>inventory</code>	Ingredient details and stock levels in two tables linked 1 : 1.
MENU_ITEM_INVENTORY	<code>menu_item_ingredients</code>	Junction table with quantity_required per serving.
FOOD_ORDER	<code>orders</code>	Adds pickup_location and pickup_lane; money lives in bills.
ORDER_ITEM	<code>order_items</code>	Line total computed, not stored.
BILL	<code>bills</code>	Adds discount; CHECK keeps <code>total_amount = subtotal + tax - discount</code> .
PAYMENT	<code>payments</code>	Adds payment_status (PENDING / SUCCESS / FAILED) to record every attempt.

6. Data Dictionary

The data dictionary documents each relation of the logical schema: column, data type (MySQL 8), key, whether NULL is allowed, constraints and meaning. Columns marked * are added by the ER-to-relational mapping.

6.1 CUSTOMER

Stores people who order food: students, faculty and other campus users.

Table 12: Data dictionary: CUSTOMER

Column	Data type	Key	Null	Constraints	Description
customer_id	INT UNSIGNED	PK	No	AUTO_INCREMENT	Unique customer identifier
name	VARCHAR(100)	—	No	—	Full name
roll_no	VARCHAR(20)	UNIQUE	Yes	NULL for non-students	University roll number
email	VARCHAR(150)	UNIQUE	No	Valid email format	Login / contact email
phone	VARCHAR(15)	UNIQUE	Yes	10–15 digits	Mobile number
user_type	ENUM	—	No	'Student','Faculty','Staff'	Customer classification
department	VARCHAR(100)	—	Yes	—	Department or school

6.2 STAFF

Stores canteen employees who run the system.

Table 13: Data dictionary: STAFF

Column	Data type	Key	Null	Constraints	Description
staff_id	INT UNSIGNED	PK	No	AUTO_INCREMENT	Unique staff identifier
name	VARCHAR(100)	—	No	—	Full name
role	ENUM	—	No	'Admin','Cashier','Cook','Manager'	Job role
phone	VARCHAR(15)	UNIQUE	Yes	10–15 digits	Mobile number
email	VARCHAR(150)	UNIQUE	No	Valid email format	Work email
username	VARCHAR(50)	UNIQUE	No	—	Login name
password_hash	VARCHAR(255)	—	No	bcrypt hash only	Hashed password

6.3 CATEGORY

Groups food items on the menu.

Table 14: Data dictionary: CATEGORY

Column	Data type	Key	Null	Constraints	Description
category_id	INT UNSIGNED	PK	No	AUTO_INCREMENT	Unique category identifier
category_name	VARCHAR(50)	UNIQUE	No	—	Category name
description	VARCHAR(255)	—	Yes	—	Short description

6.4 MENU_ITEM

Kitchen dishes managed by staff; each uses stock items.

Table 15: Data dictionary: MENU_ITEM

Column	Data type	Key	Null	Constraints	Description
menu_id	INT UNSIGNED	PK	No	AUTO_INCREMENT	Unique menu-item identifier
staff_id*	INT UNSIGNED	FK	Yes	→ STAFF; ON DELETE SET NULL	Staff member who manages it
item_name	VARCHAR(100)	UNIQUE	No	Not blank	Dish name
description	VARCHAR(255)	—	Yes	—	Short description
price	DECIMAL(8,2)	—	No	CHECK ≥ 0	Price
category	ENUM	—	No	'Veg','Non-Veg'	Dietary category
is_available	BOOLEAN	—	No	DEFAULT TRUE	Whether it can be prepared

6.5 FOOD_ITEM

Items customers can order, with category, price and availability.

Table 16: Data dictionary: FOOD_ITEM

Column	Data type	Key	Null	Constraints	Description
item_id	INT UNSIGNED	PK	No	AUTO_INCREMENT	Unique food-item identifier
category_id	INT UNSIGNED	FK	No	→ CATEGORY; ON DELETE RESTRICT	Category of the item
menu_id*	INT UNSIGNED	FK	Yes	→ MENU_ITEM; ON DELETE SET NULL	Kitchen dish it maps to
item_name	VARCHAR(100)	UNIQUE	No	Not blank	Name shown on the menu
description	VARCHAR(255)	—	Yes	—	Short description
price	DECIMAL(8,2)	—	No	CHECK ≥ 0	Current menu price
is_available	BOOLEAN	—	No	DEFAULT TRUE	Whether it can be ordered
image_url	VARCHAR(255)	—	Yes	—	Picture of the item

6.6 INVENTORY

Stock items (ingredients) with quantity and reorder level.

Table 17: Data dictionary: INVENTORY

Column	Data type	Key	Null	Constraints	Description
inventory_id	INT UNSIGNED	PK	No	AUTO_INCREMENT	Unique stock-item identifier
item_id	INT UNSIGNED	FK	Yes	→ FOOD_ITEM; ON DELETE SET NULL	Related food item
ingredient_name	VARCHAR(100)	UNIQUE	No	—	Ingredient name
quantity	DECIMAL(12,3)	—	No	CHECK ≥ 0	Quantity in stock
unit	ENUM	—	No	'g','kg','ml','l','pcs'	Unit of measure
reorder_level	DECIMAL(12,3)	—	No	CHECK ≥ 0	Low-stock threshold

6.7 MENU_ITEM_INVENTORY

Associative relation for the M : N “uses” relationship.

Table 18: Data dictionary: MENU_ITEM_INVENTORY

Column	Data type	Key	Null	Constraints	Description
menu_id*	INT UNSIGNED	PK, FK	No	→ MENU_ITEM; ON DELETE CASCADE	Menu item
inventory_id*	INT UNSIGNED	PK, FK	No	→ INVENTORY; ON DELETE RESTRICT	Stock item used

6.8 FOOD_ORDER

Order header: who ordered, when, its status and type.

Table 19: Data dictionary: FOOD_ORDER

Column	Data type	Key	Null	Constraints	Description
order_id	INT UNSIGNED	PK	No	AUTO_INCREMENT	Order / token number
customer_id	INT UNSIGNED	FK	No	→ CUSTOMER; ON DELETE RESTRICT	Customer who placed it
order_time	DATETIME	—	No	DEFAULT CURRENT_TIMESTAMP	When the order was placed
status	ENUM	—	No	'Pending','Preparing','Completed','Cancelled'	Current order state
total_amount	DECIMAL(10,2)	—	No	CHECK ≥ 0	Order amount (derived)
payment_status	ENUM	—	No	'Paid','Unpaid'	Payment state (derived)
order_type	ENUM	—	No	'Dine-in','Takeaway'	Service type

6.9 ORDER_ITEM

One row per food item in an order.

Table 20: Data dictionary: ORDER_ITEM

Column	Data type	Key	Null	Constraints	Description
order_item_id	INT UNSIGNED	PK	No	AUTO_INCREMENT	Line identifier
order_id	INT UNSIGNED	FK	No	→ FOOD_ORDER; ON DELETE CASCADE; UNIQUE with item_id	Order the line belongs to
item_id	INT UNSIGNED	FK	No	→ FOOD_ITEM; ON DELETE RESTRICT	Item ordered
quantity	SMALLINT UNSIGNED	—	No	CHECK > 0	Units ordered
unit_price	DECIMAL(8,2)	—	No	CHECK ≥ 0	Price at order time
subtotal	DECIMAL(10,2)	—	No	= quantity × unit_price	Line total (derived)

6.10 BILL

One bill per order with tax and total.

Table 21: Data dictionary: BILL

Column	Data type	Key	Null	Constraints	Description
bill_id	INT UNSIGNED	PK	No	AUTO_INCREMENT	Bill number
order_id	INT UNSIGNED	FK, UNIQUE	No	→ FOOD_ORDER; ON DELETE RESTRICT	Order being billed
bill_date	DATETIME	—	No	DEFAULT CURRENT_TIMESTAMP	Bill date and time
subtotal	DECIMAL(10,2)	—	No	CHECK ≥ 0	Sum of order lines
tax_amount	DECIMAL(10,2)	—	No	CHECK ≥ 0	GST
total_amount	DECIMAL(10,2)	—	No	CHECK = subtotal + tax_amount	Amount payable

6.11 PAYMENT

Payment attempts against a bill.

Table 22: Data dictionary: PAYMENT

Column	Data type	Key	Null	Constraints	Description
payment_id	INT UNSIGNED	PK	No	AUTO_INCREMENT	Payment identifier
bill_id	INT UNSIGNED	FK	No	→ BILL; ON DELETE RESTRICT	Bill being paid
payment_time	DATETIME	—	No	DEFAULT CURRENT_TIMESTAMP	When the payment was made
amount	DECIMAL(10,2)	—	No	CHECK > 0	Amount paid
payment_method	ENUM	—	No	'Cash','Card','UPI'	Payment method
transaction_id	VARCHAR(64)	UNIQUE	Yes	Required unless Cash	Gateway / UPI reference

7. SQL Commands Used (DDL and DML) with Sample Outputs

The database `canteen_db` is created by a single script, `database.sql`, in the project repository. Running `mysql -u root -p < database.sql` drops and recreates the database, creates all tables and the view, and loads the sample data. All statements and outputs in this chapter are taken from that script and from the running MySQL 8.0 server.

7.1 Physical Schema Overview

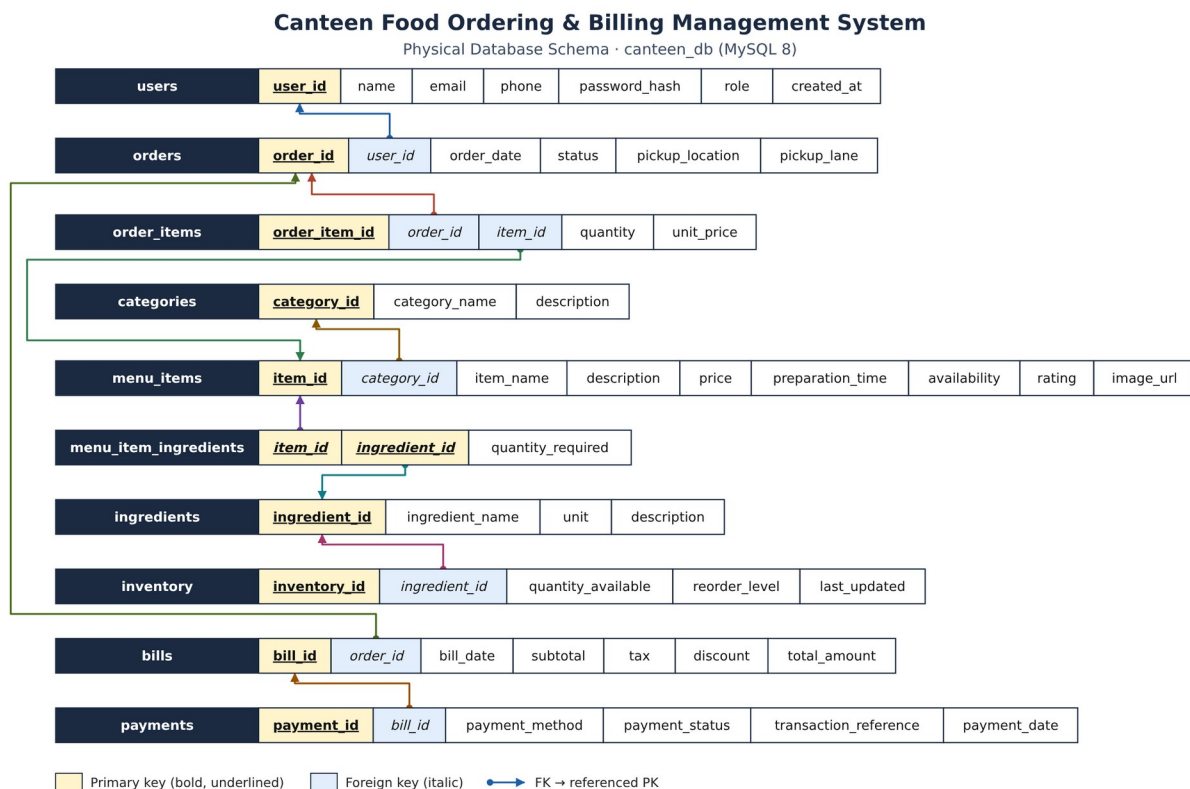


Figure 3: Physical database schema of `canteen_db` as implemented in `database.sql`

```
mysql> SHOW FULL TABLES;
```

```
+-----+-----+
| Tables_in_canteen_db | Table_type |
+-----+-----+
| bills                 | BASE TABLE |
| categories            | BASE TABLE |
| ingredients           | BASE TABLE |
| inventory             | BASE TABLE |
| menu_item_ingredients | BASE TABLE |
| menu_items           | BASE TABLE |
| order_items          | BASE TABLE |
| orders               | BASE TABLE |
| payments             | BASE TABLE |
| users                | BASE TABLE |
| v_order_summary      | VIEW        |
+-----+-----+
```

7.2 Data Definition Language (DDL)

Each table is created with named constraints so that error messages identify exactly which rule was broken. The complete DDL from `database.sql` is shown below.

Creating the database

```
DROP DATABASE IF EXISTS canteen_db;
CREATE DATABASE canteen_db
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_0900_ai_ci;
USE canteen_db;
```

Table users

```
CREATE TABLE users (
    user_id      INT UNSIGNED NOT NULL AUTO_INCREMENT,
    name         VARCHAR(100) NOT NULL,
    email        VARCHAR(150) NOT NULL,
    phone        VARCHAR(15)  NULL,
    password_hash VARCHAR(255) NOT NULL,           -- bcrypt hash only, never plaintext
    role         ENUM('STUDENT','ADMIN','STAFF') NOT NULL DEFAULT 'STUDENT',
    created_at   TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT pk_users      PRIMARY KEY (user_id),
    CONSTRAINT uq_users_email UNIQUE (email),
    CONSTRAINT uq_users_phone UNIQUE (phone),
    CONSTRAINT chk_users_email CHECK (email LIKE '%_@_._%'),
    CONSTRAINT chk_users_phone CHECK (phone IS NULL OR phone REGEXP '^[0-9+][0-9]{9,14}$'),
    CONSTRAINT chk_users_hash CHECK (CHAR_LENGTH(password_hash) >= 50)
) ENGINE=InnoDB;
```

Table categories

```
CREATE TABLE categories (  
  category_id INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  category_name VARCHAR(50) NOT NULL,  
  description VARCHAR(255) NULL,  
  CONSTRAINT pk_categories PRIMARY KEY (category_id),  
  CONSTRAINT uq_categories_name UNIQUE (category_name)  
) ENGINE=InnoDB;
```

Table menu_items

```
CREATE TABLE menu_items (  
  item_id INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  category_id INT UNSIGNED NOT NULL,  
  item_name VARCHAR(100) NOT NULL,  
  description VARCHAR(255) NULL,  
  price DECIMAL(8,2) NOT NULL,  
  preparation_time SMALLINT UNSIGNED NOT NULL DEFAULT 10, -- minutes  
  availability BOOLEAN NOT NULL DEFAULT TRUE,  
  rating DECIMAL(2,1) NULL,  
  image_url VARCHAR(255) NULL,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT pk_menu_items PRIMARY KEY (item_id),  
  CONSTRAINT uq_menu_items_name UNIQUE (item_name),  
  CONSTRAINT fk_menu_items_cat FOREIGN KEY (category_id)  
    REFERENCES categories (category_id)  
    ON UPDATE CASCADE ON DELETE RESTRICT,  
  CONSTRAINT chk_menu_price CHECK (price >= 0),  
  CONSTRAINT chk_menu_prep_time CHECK (preparation_time >= 0),  
  CONSTRAINT chk_menu_rating CHECK (rating IS NULL OR rating BETWEEN 0 AND 5),  
  CONSTRAINT chk_menu_name CHECK (CHAR_LENGTH(TRIM(item_name)) > 0)  
) ENGINE=InnoDB;
```

Table ingredients

```
CREATE TABLE ingredients (  
  ingredient_id INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  ingredient_name VARCHAR(100) NOT NULL,  
  unit ENUM('g','kg','ml','l','pcs') NOT NULL,  
  description VARCHAR(255) NULL,  
  CONSTRAINT pk_ingredients PRIMARY KEY (ingredient_id),  
  CONSTRAINT uq_ingredients_name UNIQUE (ingredient_name)  
) ENGINE=InnoDB;
```

Table menu_item_ingredients

```
CREATE TABLE menu_item_ingredients (  
    item_id            INT UNSIGNED    NOT NULL,  
    ingredient_id      INT UNSIGNED    NOT NULL,  
    quantity_required  DECIMAL(10,3)  NOT NULL,    -- per 1 serving, in ingredient's unit  
    CONSTRAINT pk_menu_item_ingredients PRIMARY KEY (item_id, ingredient_id),  
    CONSTRAINT fk_mii_item FOREIGN KEY (item_id)  
        REFERENCES menu_items (item_id)  
        ON UPDATE CASCADE ON DELETE CASCADE,  
    CONSTRAINT fk_mii_ingredient FOREIGN KEY (ingredient_id)  
        REFERENCES ingredients (ingredient_id)  
        ON UPDATE CASCADE ON DELETE RESTRICT,  
    CONSTRAINT chk_mii_qty CHECK (quantity_required > 0)  
) ENGINE=InnoDB;
```

Table inventory

```
CREATE TABLE inventory (  
    inventory_id       INT UNSIGNED    NOT NULL AUTO_INCREMENT,  
    ingredient_id      INT UNSIGNED    NOT NULL,  
    quantity_available DECIMAL(12,3)  NOT NULL DEFAULT 0,  
    reorder_level      DECIMAL(12,3)  NOT NULL DEFAULT 0,  
    last_updated       TIMESTAMP       NOT NULL DEFAULT CURRENT_TIMESTAMP  
        ON UPDATE CURRENT_TIMESTAMP,  
    CONSTRAINT pk_inventory PRIMARY KEY (inventory_id),  
    CONSTRAINT uq_inventory_ingredient UNIQUE (ingredient_id),  
    CONSTRAINT fk_inventory_ingredient FOREIGN KEY (ingredient_id)  
        REFERENCES ingredients (ingredient_id)  
        ON UPDATE CASCADE ON DELETE CASCADE,  
    CONSTRAINT chk_inventory_qty CHECK (quantity_available >= 0),  
    CONSTRAINT chk_inventory_reorder CHECK (reorder_level >= 0)  
) ENGINE=InnoDB;
```

Table orders

```
CREATE TABLE orders (  
  order_id      INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  user_id       INT UNSIGNED NOT NULL,  
  order_date    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  status        ENUM('PENDING', 'CONFIRMED', 'PREPARING', 'READY', 'COMPLETED', 'CANCELLED')  
               NOT NULL DEFAULT 'PENDING',  
  pickup_location VARCHAR(100) NOT NULL DEFAULT 'Main Canteen',  
  pickup_lane   VARCHAR(20) NULL,  
  CONSTRAINT pk_orders PRIMARY KEY (order_id),  
  CONSTRAINT fk_orders_user FOREIGN KEY (user_id)  
    REFERENCES users (user_id)  
    ON UPDATE CASCADE ON DELETE RESTRICT,  
  INDEX idx_orders_user_date (user_id, order_date),  
  INDEX idx_orders_status (status)  
) ENGINE=InnoDB;
```

Table order_items

```
CREATE TABLE order_items (  
  order_item_id INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  order_id      INT UNSIGNED NOT NULL,  
  item_id       INT UNSIGNED NOT NULL,  
  quantity      SMALLINT UNSIGNED NOT NULL DEFAULT 1,  
  unit_price    DECIMAL(8,2) NOT NULL,  
  CONSTRAINT pk_order_items PRIMARY KEY (order_item_id),  
  CONSTRAINT uq_order_items_order_item UNIQUE (order_id, item_id),  
  CONSTRAINT fk_order_items_order FOREIGN KEY (order_id)  
    REFERENCES orders (order_id)  
    ON UPDATE CASCADE ON DELETE CASCADE,  
  CONSTRAINT fk_order_items_item FOREIGN KEY (item_id)  
    REFERENCES menu_items (item_id)  
    ON UPDATE CASCADE ON DELETE RESTRICT,  
  CONSTRAINT chk_order_items_qty CHECK (quantity > 0),  
  CONSTRAINT chk_order_items_price CHECK (unit_price >= 0)  
) ENGINE=InnoDB;
```

Table bills

```
CREATE TABLE bills (  
    bill_id      INT UNSIGNED NOT NULL AUTO_INCREMENT,  
    order_id     INT UNSIGNED NOT NULL,  
    bill_date    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    subtotal     DECIMAL(10,2) NOT NULL,  
    tax          DECIMAL(10,2) NOT NULL DEFAULT 0,  
    discount     DECIMAL(10,2) NOT NULL DEFAULT 0,  
    total_amount DECIMAL(10,2) NOT NULL,  
    CONSTRAINT pk_bills PRIMARY KEY (bill_id),  
    CONSTRAINT uq_bills_order UNIQUE (order_id),  
    CONSTRAINT fk_bills_order FOREIGN KEY (order_id)  
        REFERENCES orders (order_id)  
        ON UPDATE CASCADE ON DELETE RESTRICT,  
    CONSTRAINT chk_bills_subtotal CHECK (subtotal >= 0),  
    CONSTRAINT chk_bills_tax CHECK (tax >= 0),  
    CONSTRAINT chk_bills_discount CHECK (discount >= 0 AND discount <= subtotal + tax),  
    CONSTRAINT chk_bills_total CHECK (total_amount = subtotal + tax - discount)  
) ENGINE=InnoDB;
```

Table payments

```
CREATE TABLE payments (  
    payment_id    INT UNSIGNED NOT NULL AUTO_INCREMENT,  
    bill_id       INT UNSIGNED NOT NULL,  
    payment_method ENUM('UPI','CAMPUS_WALLET','CASH') NOT NULL,  
    payment_status ENUM('PENDING','SUCCESS','FAILED') NOT NULL DEFAULT 'PENDING',  
    transaction_reference VARCHAR(64) NULL, -- NULL allowed for cash  
    payment_date  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT pk_payments PRIMARY KEY (payment_id),  
    CONSTRAINT uq_payments_txnref UNIQUE (transaction_reference),  
    CONSTRAINT fk_payments_bill FOREIGN KEY (bill_id)  
        REFERENCES bills (bill_id)  
        ON UPDATE CASCADE ON DELETE RESTRICT,  
    CONSTRAINT chk_payments_txnref CHECK (  
        payment_method = 'CASH' OR transaction_reference IS NOT NULL)  
) ENGINE=InnoDB;
```

View v_order_summary

```
CREATE VIEW v_order_summary AS
SELECT  o.order_id,
        o.user_id,
        o.order_date,
        o.status,
        o.pickup_location,
        o.pickup_lane,
        SUM(oi.quantity * oi.unit_price) AS items_subtotal,
        b.bill_id,
        b.tax,
        b.discount,
        b.total_amount
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
LEFT JOIN bills b ON b.order_id = o.order_id
GROUP BY o.order_id, o.user_id, o.order_date, o.status, o.pickup_location,
         o.pickup_lane, b.bill_id, b.tax, b.discount, b.total_amount;
```

7.3 Data Manipulation Language (DML)

Sample data is inserted with multi-row `INSERT` statements. Excerpts are shown below; the full statements are in `database.sql`.

Users (passwords stored only as bcrypt hashes)

```
INSERT INTO users (user_id, name, email, phone, password_hash, role, created_at) VALUES
(1, 'Rajesh Kumar', 'admin@canteen.woxsen.edu.in', '9876500001', '$2b$10$ctEBkLLS...', 'ADMIN', '2026-07-01 09:00:00'),
(2, 'Sunita Reddy', 'sunita.staff@canteen.woxsen.edu.in', '9876500002', '$2b$10$3vrFxsFX...', 'STAFF', '2026-07-01 09:30:00'),
-- ... 4 more rows ('Aarav Sharma', 'Priya Iyer', 'Rohan Verma', 'Ananya Gupta')
-- (bcrypt hashes are 60 characters long; shortened here for readability)
```

Categories and menu items

```
INSERT INTO categories (category_id, category_name, description) VALUES
(1, 'Burgers & Wraps', 'Burgers, wraps and rolls'),
(2, 'Main Course',    'Biryani, rice and noodle meals'),
(3, 'South Indian',   'Dosa, idli and other South Indian dishes'),
(4, 'Snacks',         'Quick bites and fried snacks'),
(5, 'Beverages',      'Hot and cold drinks'),
(6, 'Desserts',       'Sweets and baked treats');

INSERT INTO menu_items (item_id, category_id, item_name, description, price, preparation_time, availability,
rating, image_url) VALUES
(1, 1, 'Chicken Burger', 'Crispy chicken patty with lettuce and mayo',    120.00, 12, TRUE, 4.4,
'/images/chicken-burger.jpg'),
(2, 1, 'Veg Burger',     'Aloo-veg patty with onion, tomato and sauce',    80.00, 10, TRUE, 4.1,
'/images/veg-burger.jpg'),
(3, 1, 'Paneer Wrap',    'Tandoori paneer tikka rolled in a soft roti',    110.00, 10, TRUE, 4.5,
'/images/paneer-wrap.jpg'),
-- ... 10 more rows (Brownie is inserted with availability = FALSE)
```

Ingredients, recipes and stock

```
INSERT INTO ingredients (ingredient_id, ingredient_name, unit, description) VALUES
(1, 'Chicken',        'g', 'Boneless chicken'),
(2, 'Paneer',         'g', 'Fresh cottage cheese'),
-- ... 16 more ingredients

INSERT INTO menu_item_ingredients (item_id, ingredient_id, quantity_required) VALUES
(1, 1, 120), (1, 3, 1), (1, 6, 20), (1, 7, 20), (1, 15, 30),      -- Chicken Burger
(2, 5, 100), (2, 3, 1), (2, 6, 20), (2, 15, 30),                -- Veg Burger
-- ... recipes for the remaining 11 items

INSERT INTO inventory (ingredient_id, quantity_available, reorder_level) VALUES
(1, 15000, 5000), -- Chicken (g)
(2, 6000, 2000),  -- Paneer (g)
-- ... 16 more stock rows
```

Orders and order lines

```
INSERT INTO orders (order_id, user_id, order_date, status, pickup_location, pickup_lane) VALUES
(1, 3, '2026-10-01 12:45:00', 'COMPLETED', 'Main Canteen', 'Lane 1'),
(2, 4, '2026-10-01 13:10:00', 'COMPLETED', 'Main Canteen', 'Lane 2'),
(3, 5, '2026-10-02 16:30:00', 'COMPLETED', 'Food Court', 'Lane 1'),
(4, 6, '2026-10-03 09:15:00', 'CANCELLED', 'Main Canteen', 'Lane 3'),
(5, 3, '2026-10-05 13:00:00', 'READY', 'Main Canteen', 'Lane 1'),
(6, 4, '2026-10-06 12:20:00', 'PREPARING', 'Food Court', 'Lane 2'),
(7, 5, '2026-10-06 12:40:00', 'PENDING', 'Main Canteen', NULL);

INSERT INTO order_items (order_id, item_id, quantity, unit_price) VALUES
(1, 4, 1, 160.00), (1, 11, 2, 20.00), -- Aarav
(2, 3, 1, 110.00), (2, 10, 1, 60.00), (2, 9, 2, 30.00), -- Priya
(3, 1, 2, 120.00), (3, 8, 1, 60.00), (3, 12, 2, 40.00), -- Rohan
(4, 7, 1, 70.00), (4, 11, 1, 20.00), -- Ananya (cancelled)
(5, 5, 1, 120.00), (5, 13, 1, 70.00), -- Aarav
(6, 6, 2, 90.00), (6, 12, 1, 40.00), -- Priya
(7, 2, 1, 80.00), (7, 11, 1, 20.00); -- Rohan
```

Bills computed from the order lines (INSERT ... SELECT)

Bills are not typed in by hand. The subtotal is calculated from `order_items` inside the INSERT, so it can never disagree with the order lines, and 5% GST is applied.

```
INSERT INTO bills (order_id, bill_date, subtotal, tax, discount, total_amount)
SELECT t.order_id, t.order_date, t.sub, t.tax, t.disc, t.sub + t.tax - t.disc
FROM (
    SELECT o.order_id,
           o.order_date,
           SUM(oi.quantity * oi.unit_price) AS sub,
           ROUND(SUM(oi.quantity * oi.unit_price) * 0.05, 2) AS tax,
           CASE o.order_id WHEN 3 THEN 20.00 WHEN 2 THEN 10.00 ELSE 0.00 END AS disc
    FROM orders o
    JOIN order_items oi ON oi.order_id = o.order_id
    WHERE o.status NOT IN ('CANCELLED', 'PENDING')
    GROUP BY o.order_id, o.order_date
) t
ORDER BY t.order_id;
-- Resulting bill_ids: 1->order 1, 2->order 2, 3->order 3, 4->order 5, 5->order 6
```

Payments (one bill can have several attempts)

```
INSERT INTO payments (bill_id, payment_method, payment_status, transaction_reference, payment_date) VALUES
(1, 'UPI', 'SUCCESS', 'UPI2026100112451001', '2026-10-01 12:46:00'),
(2, 'CAMPUS_WALLET', 'SUCCESS', 'CW-2026100113100002', '2026-10-01 13:11:00'),
(3, 'UPI', 'FAILED', 'UPI2026100216300003', '2026-10-02 16:31:00'),
(3, 'CASH', 'SUCCESS', NULL, '2026-10-02 16:33:00'),
(4, 'UPI', 'SUCCESS', 'UPI2026100513000004', '2026-10-05 13:01:00'),
(5, 'CAMPUS_WALLET', 'PENDING', 'CW-2026100612200005', '2026-10-06 12:21:00');
```

7.4 Transactional Order Placement

Placing an order touches five tables. The application runs all of the steps on one connection inside a transaction: if any step fails (for example, not enough stock, or an item has just been marked sold out), **ROLLBACK** undoes everything and no partial order is stored. The statements executed for one order are:

SQL executed by POST /api/orders (server.js)

```
START TRANSACTION;
-- 1. prices and availability come from the database, not the browser
SELECT item_id, item_name, price, availability FROM menu_items WHERE item_id IN (?);
-- 2. order header
INSERT INTO orders (user_id, status, pickup_location, pickup_lane) VALUES (?, 'PENDING', ?, ?);
-- 3. one line per item, at the current price
INSERT INTO order_items (order_id, item_id, quantity, unit_price) VALUES (?, ?, ?, ?), ...;
-- 4. lock the stock rows this order needs, check them, then deduct
SELECT inv.inventory_id, SUM(mii.quantity_required * oi.quantity) AS needed
  FROM order_items oi JOIN menu_item_ingredients mii ON ... JOIN inventory inv ON ...
 WHERE oi.order_id = ? GROUP BY ... FOR UPDATE;
UPDATE inventory SET quantity_available = quantity_available - ? WHERE inventory_id = ?;
-- 5. bill with 5% GST
INSERT INTO bills (order_id, subtotal, tax, discount, total_amount) VALUES (?, ?, ?, 0, ?);
-- 6. payment attempt
INSERT INTO payments (bill_id, payment_method, payment_status, transaction_reference)
VALUES (?, ?, ?, ?);
UPDATE orders SET status = 'CONFIRMED' WHERE order_id = ?;   -- UPI / campus wallet only
COMMIT;                                                       -- or ROLLBACK on any error
```

Cancelling an order (by the student before preparation, or by staff at any stage before completion) runs a similar transaction that sets the status to **CANCELLED** and adds the same ingredient quantities back to **inventory**.

7.5 Sample Data Overview

Table 23: Rows loaded by database.sql

Table	Rows	Notable sample data
users	6	1 admin, 1 staff member, 4 students
categories	6	Burgers & Wraps, Main Course, South Indian, Snacks, Beverages, Desserts
menu_items	13	Brownie is marked sold out
ingredients	18	Units g, ml and pcs
menu_item_ingredients	43	Every menu item has a recipe
inventory	18	Noodles and Chocolate are below their reorder level
orders	7	3 completed, 1 cancelled, 1 ready, 1 preparing, 1 pending
order_items	16	2-3 lines per order
bills	5	Two bills carry a discount
payments	6	Bill 3 has a failed UPI attempt followed by a successful cash payment

8. Queries with Outputs

The following queries were executed against `canteen_db` immediately after loading `database.sql`. The outputs are the actual result sets returned by MySQL 8.0.

8.1 Join query: customer order history

Query 1

```
SELECT u.name AS customer, o.order_id, o.status, b.total_amount
FROM users u
JOIN orders o      ON o.user_id = u.user_id
LEFT JOIN bills b  ON b.order_id = o.order_id
ORDER BY o.order_id;
```

Table 24: Output of Query 1

customer	order_id	status	total_amount
Aarav Sharma	1	COMPLETED	210.00
Priya Iyer	2	COMPLETED	231.50
Rohan Verma	3	COMPLETED	379.00
Ananya Gupta	4	CANCELLED	NULL
Aarav Sharma	5	READY	199.50
Priya Iyer	6	PREPARING	231.00
Rohan Verma	7	PENDING	NULL

An inner join with `users` and a left join with `bills` lists every order with its customer; orders without a bill (cancelled or pending) show NULL for the amount.

8.2 Aggregate query: revenue by category

Query 2

```
SELECT c.category_name,
       SUM(oi.quantity * oi.unit_price) AS revenue
FROM categories c
JOIN menu_items m  ON m.category_id = c.category_id
JOIN order_items oi ON oi.item_id    = m.item_id
JOIN orders o      ON o.order_id     = oi.order_id
WHERE o.status <> 'CANCELLED'
GROUP BY c.category_id, c.category_name
ORDER BY revenue DESC;
```

Table 25: Output of Query 2

category_name	revenue
Main Course	460.00
Burgers & Wraps	430.00
Beverages	240.00
Snacks	120.00
Desserts	70.00

Line totals are calculated from quantity × unit_price and grouped by category; cancelled orders are excluded.

8.3 Aggregate query: paid bills, revenue and average bill value

Query 3

```
SELECT COUNT(*)           AS paid_bills,
       SUM(b.total_amount) AS revenue,
       ROUND(AVG(b.total_amount),2) AS average_bill
FROM bills b
WHERE EXISTS (SELECT 1 FROM payments p
              WHERE p.bill_id = b.bill_id AND p.payment_status = 'SUCCESS');
```

Table 26: Output of Query 3

paid_bills	revenue	average_bill
4	1020.00	255.00

A correlated EXISTS subquery restricts the aggregate to bills that have at least one successful payment.

8.4 Popular food items

Query 4

```
SELECT m.item_name, SUM(oi.quantity) AS units_sold
FROM menu_items m
JOIN order_items oi ON oi.item_id = m.item_id
JOIN orders o      ON o.order_id = oi.order_id
WHERE o.status <> 'CANCELLED'
GROUP BY m.item_id, m.item_name
ORDER BY units_sold DESC, m.item_name
LIMIT 8;
```

Table 27: Output of Query 4

item_name	units_sold
Lime Soda	3
Masala Chai	3
Chicken Burger	2
Hakka Noodles	2
Samosa	2
Brownie	1
Chicken Biryani	1
Cold Coffee	1

Units sold per item across all non-cancelled orders.

8.5 Active (not yet collected) orders

Query 5

```
SELECT o.order_id, u.name AS customer, o.status,
       o.pickup_location, o.pickup_lane
FROM orders o
JOIN users u ON u.user_id = o.user_id
WHERE o.status IN ('PENDING', 'CONFIRMED', 'PREPARING', 'READY')
ORDER BY o.order_date;
```

Table 28: Output of Query 5

order_id	customer	status	pickup_location	pickup_lane
5	Aarav Sharma	READY	Main Canteen	Lane 1
6	Priya Iyer	PREPARING	Food Court	Lane 2
7	Rohan Verma	PENDING	Main Canteen	NULL

Orders the kitchen still has to finish, oldest first.

8.6 Ingredients at or below reorder level

Query 6

```
SELECT ing.ingredient_name, inv.quantity_available,
       inv.reorder_level, ing.unit
FROM inventory inv
JOIN ingredients ing ON ing.ingredient_id = inv.ingredient_id
WHERE inv.quantity_available <= inv.reorder_level;
```

Table 29: Output of Query 6

ingredient_name	quantity_available	reorder_level	unit
Noodles	1800.000	2000.000	g
Chocolate	300.000	500.000	g

The low-stock rule used by the dashboard and the inventory screen.

8.7 Payment attempts by method and status

Query 7

```
SELECT payment_method, payment_status, COUNT(*) AS attempts
FROM payments
GROUP BY payment_method, payment_status
ORDER BY payment_method, payment_status;
```

Table 30: Output of Query 7

payment_method	payment_status	attempts
UPI	SUCCESS	2
UPI	FAILED	1
CAMPUS_WALLET	PENDING	1
CAMPUS_WALLET	SUCCESS	1
CASH	SUCCESS	1

Every payment attempt is kept, so failed and pending attempts are visible alongside successful ones.

8.8 Recipe of a menu item (M:N join through MENU_ITEM_INGREDIENTS)

Query 8

```
SELECT m.item_name, ing.ingredient_name,
       mii.quantity_required, ing.unit
FROM menu_item_ingredients mii
JOIN menu_items m ON m.item_id = mii.item_id
JOIN ingredients ing ON ing.ingredient_id = mii.ingredient_id
WHERE m.item_name = 'Chicken Biryani';
```

Table 31: Output of Query 8

item_name	ingredient_name	quantity_required	unit
Chicken Biryani	Chicken	150.000	g
Chicken Biryani	Basmati Rice	200.000	g
Chicken Biryani	Onion	50.000	g
Chicken Biryani	Cooking Oil	20.000	ml

A three-table join through the junction table shows the recipe used for stock deduction.

8.9 Subquery: menu items priced above the average menu price

Query 9

```
SELECT item_name, price
FROM menu_items
WHERE price > (SELECT AVG(price) FROM menu_items)
ORDER BY price DESC;
```

Table 32: Output of Query 9

item_name	price
Chicken Biryani	160.00
Chicken Burger	120.00
Veg Biryani	120.00
Paneer Wrap	110.00
Hakka Noodles	90.00
Veg Burger	80.00

A scalar subquery computes the average menu price, which the outer query compares against.

8.10 View: v_order_summary

Query 10

```
SELECT order_id, status, items_subtotal, tax, discount, total_amount
FROM v_order_summary
ORDER BY order_id;
```

Table 33: Output of Query 10

order_id	status	items_subtotal	tax	discount	total_amount
1	COMPLETED	200.00	10.00	0.00	210.00
2	COMPLETED	230.00	11.50	10.00	231.50
3	COMPLETED	380.00	19.00	20.00	379.00
4	CANCELLED	90.00	NULL	NULL	NULL
5	READY	190.00	9.50	0.00	199.50
6	PREPARING	220.00	11.00	0.00	231.00
7	PENDING	100.00	NULL	NULL	NULL

The view combines orders, their computed subtotal and their bill, without storing any line totals.

9. UI Design and Screenshots

9.1 Design Approach

The interface is a single-page browser application written in plain HTML, CSS and JavaScript. After login, the navigation shows only the screens a user's role allows: students see **Menu** and **My orders**; staff see **Dashboard**, **Orders**, **Inventory** and **Billing**; administrators additionally see **Menu** management. Validation messages appear next to the form that caused them, destructive actions ask for confirmation, and every action ends with a short success or error message.

The most important DBMS principle shown by the UI is persistence: every screen is rendered from data returned by the API, which reads from MySQL, and every insert, update or delete made in the UI is written to MySQL. Section 9.3 shows the same records in the UI and in the database. All screenshots in this chapter were captured from the running application connected to `canteen_db`.

9.2 Screens

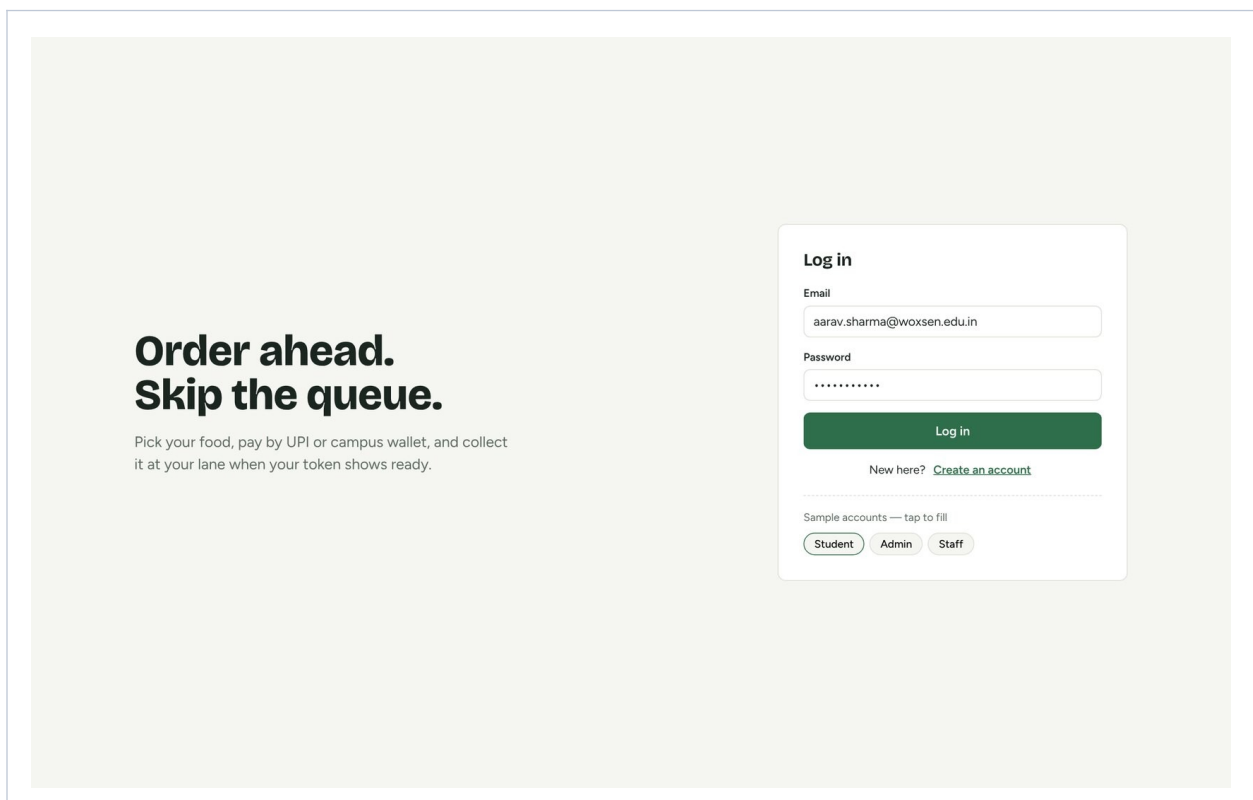


Figure 4: Login screen with sample accounts for each role

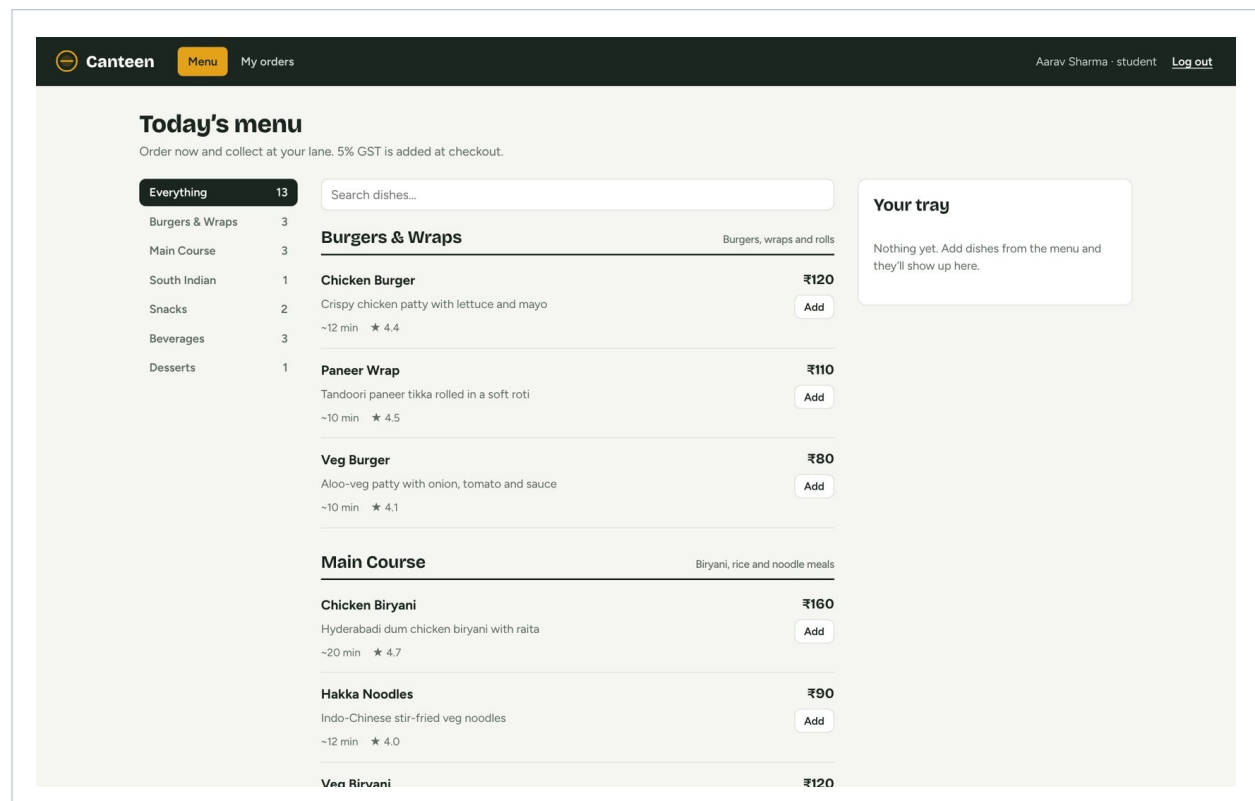


Figure 5: Student menu: categories, search, prices, preparation time and ratings

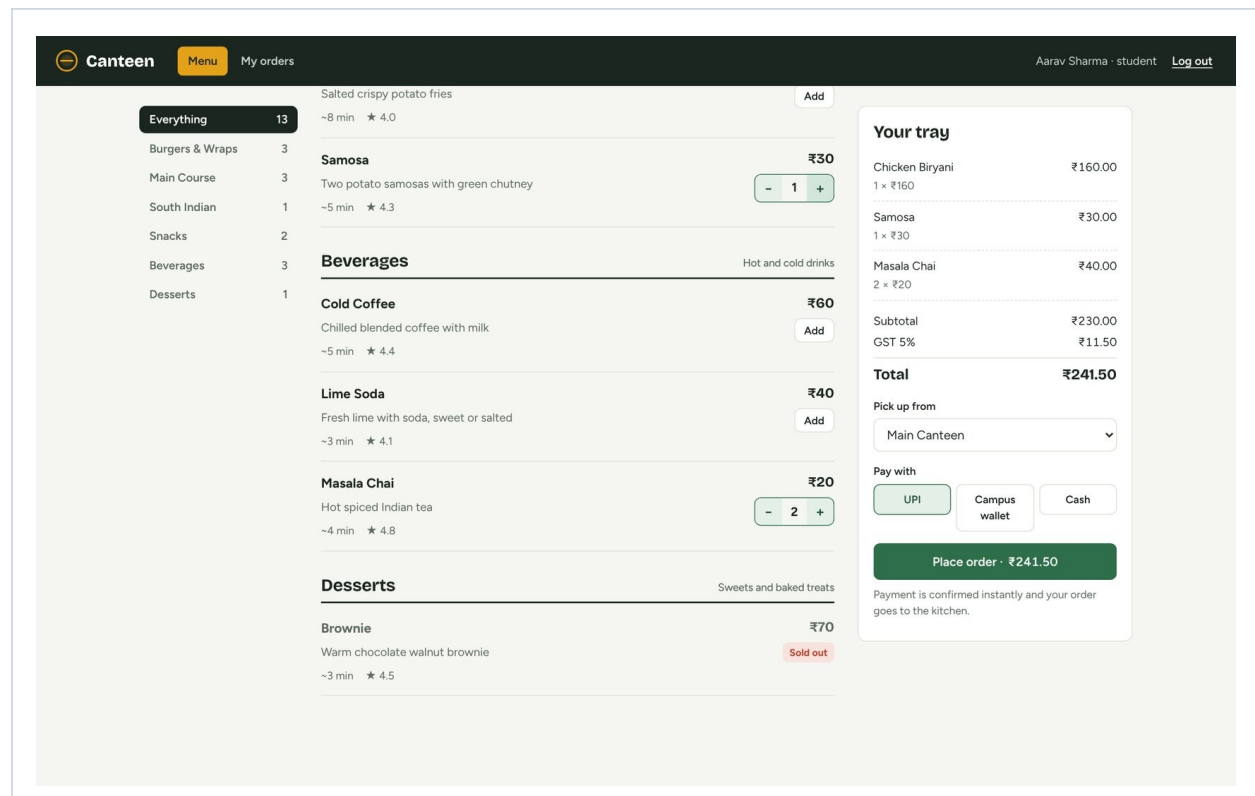


Figure 6: Tray and checkout: line totals, 5% GST, pickup point and payment method; sold-out items cannot be added

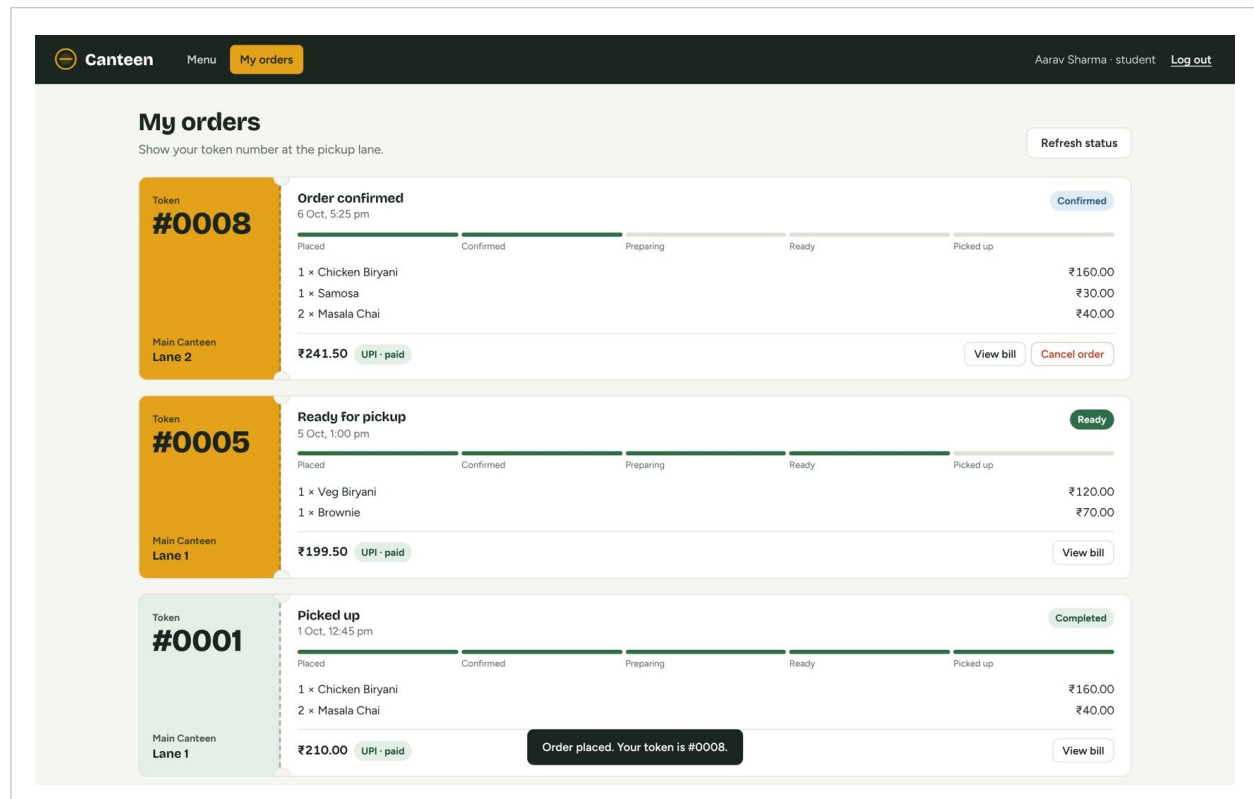


Figure 7: My orders: pickup token, lane, status progress and payment state after placing an order

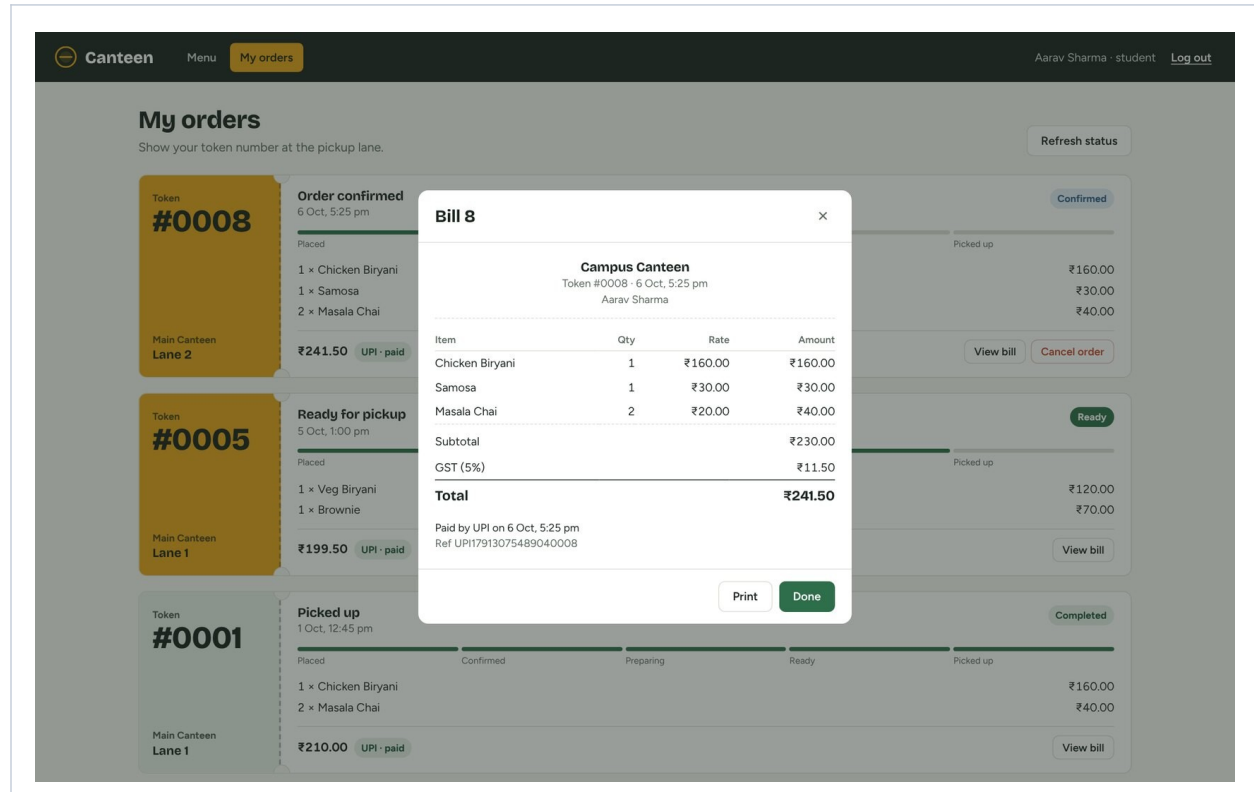


Figure 8: Bill generated for the order, with GST, total and UPI transaction reference

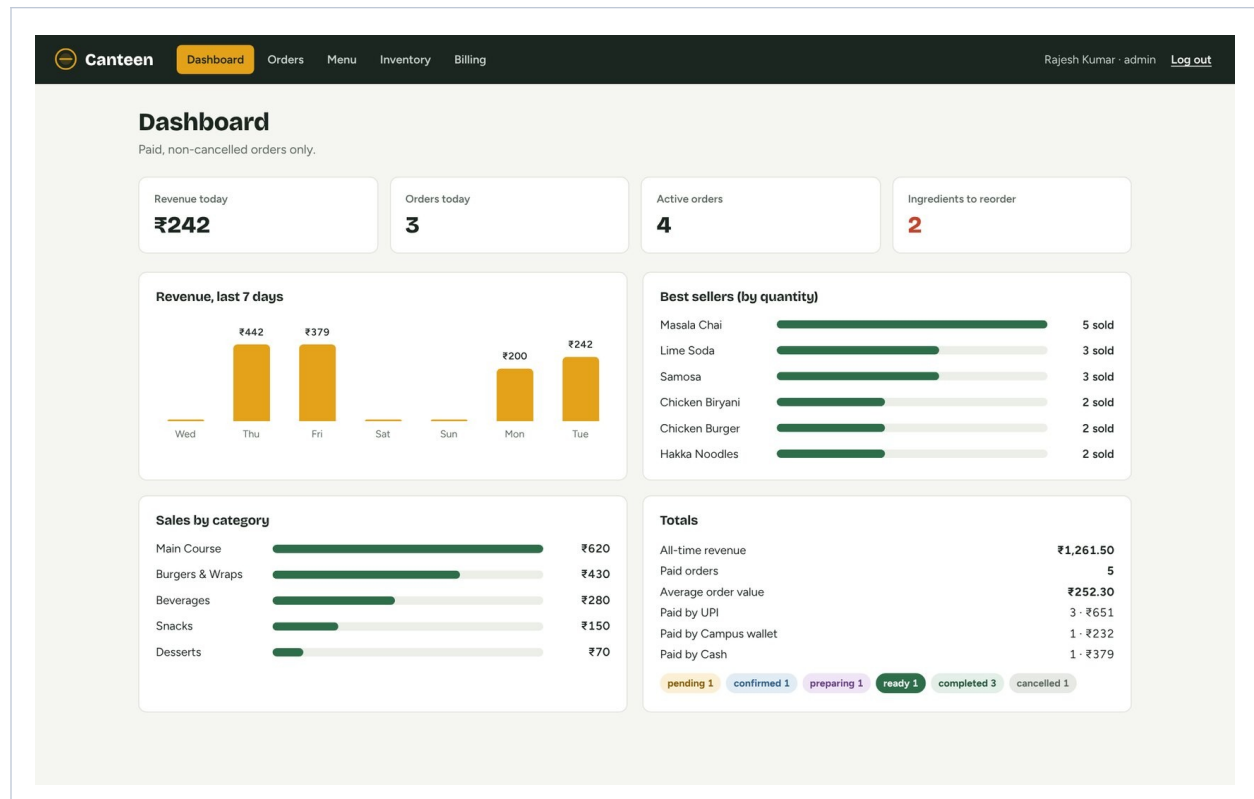


Figure 9: Staff / admin dashboard: revenue, active orders, low stock, best sellers and sales by category

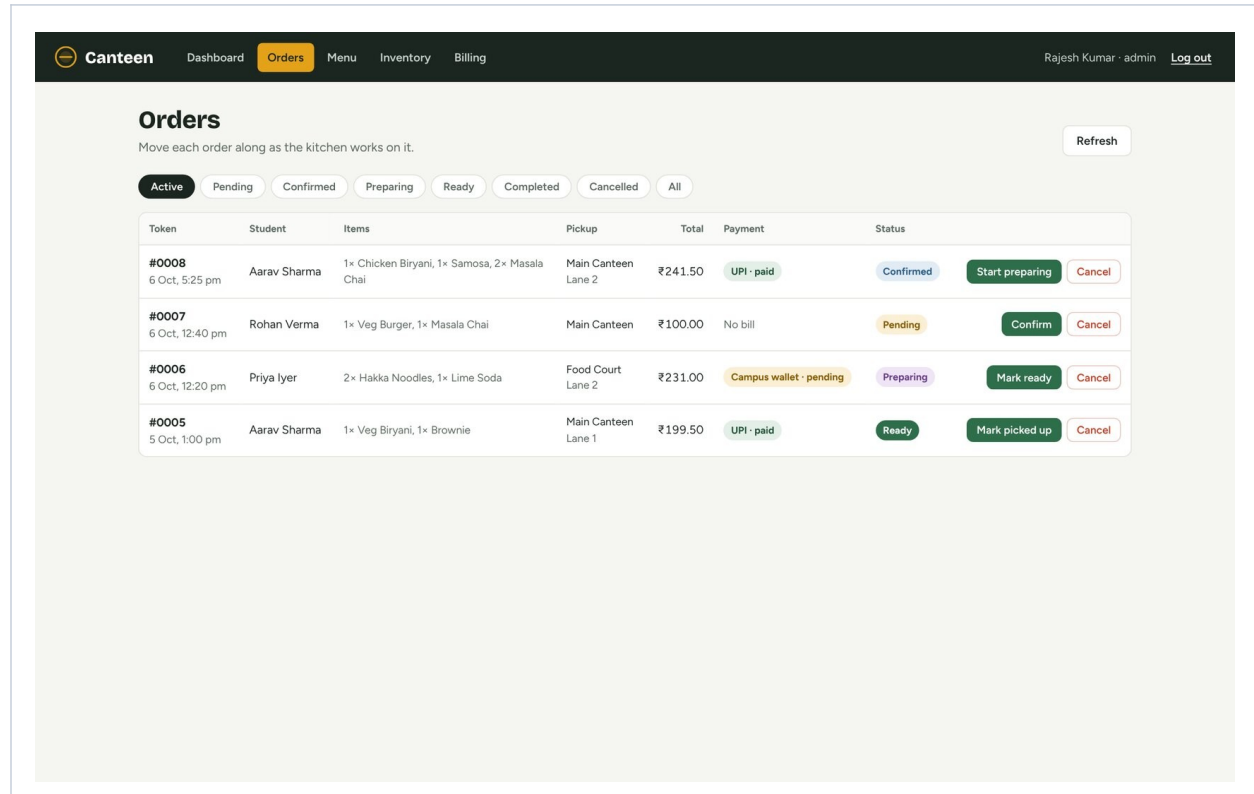


Figure 10: Order management: active orders with payment state and the next status action

Canteen

Dashboard
Orders
Menu
Inventory
Billing

Rajesh Kumar · admin
Log out

Menu

Turn items off when they run out; students see them as sold out.

Add categoryAdd item

Item	Category	Price	Prep	Rating	On menu	
Chicken Burger Crispy chicken patty with lettuce and mayo	Burgers & Wraps	₹120	12 min	4.4	☒	Edit Delete
Paneer Wrap Tandoori paneer tikka rolled in a soft roti	Burgers & Wraps	₹110	10 min	4.5	☒	Edit Delete
Veg Burger Aloo-veg patty with onion, tomato and sauce	Burgers & Wraps	₹80	10 min	4.1	☒	Edit Delete
Chicken Biryani Hyderabadi dum chicken biryani with raita	Main Course	₹160	20 min	4.7	☒	Edit Delete
Hakka Noodles Indo-Chinese stir-fried veg noodles	Main Course	₹90	12 min	4.0	☒	Edit Delete
Veg Biryani Mixed vegetable dum biryani with raita	Main Course	₹120	18 min	4.2	☒	Edit Delete
Masala Dosa Crisp dosa with potato masala, chutney, sambar	South Indian	₹70	10 min	4.6	☒	Edit Delete
French Fries Salted crispy potato fries	Snacks	₹60	8 min	4.0	☒	Edit Delete
Samosa Two potato samosas with green chutney	Snacks	₹30	5 min	4.3	☒	Edit Delete
Cold Coffee Chilled blended coffee with milk	Beverages	₹60	5 min	4.4	☒	Edit Delete
Lime Soda Fresh lime with soda, sweet or salted	Beverages	₹40	3 min	4.1	☒	Edit Delete

Figure 11: Menu management (admin): items with availability toggle, edit and delete; categories below

Canteen

Dashboard
Orders
Menu
Inventory
Billing

Rajesh Kumar · admin
Log out

Inventory

2 ingredients are at or below the reorder level. Stock is deducted automatically when orders are placed.

Ingredient	Used in	In stock	Reorder at	Status	
Chocolate Updated 6 Oct, 5:25 pm	Brownie	300 g	500 g	Reorder	Save
Noodles Updated 6 Oct, 5:25 pm	Hakka Noodles	1800 g	2000 g	Reorder	Save
Basmati Rice Updated 6 Oct, 5:25 pm	Chicken Biryani, Veg Biryani	24800 g	8000 g	OK	Save
Burger Bun Updated 6 Oct, 5:25 pm	Chicken Burger, Veg Burger	120 pcs	40 pcs	OK	Save
Chicken Updated 6 Oct, 5:25 pm	Chicken Biryani, Chicken Burger	14850 g	5000 g	OK	Save
Coffee Powder Updated 6 Oct, 5:25 pm	Cold Coffee	800 g	300 g	OK	Save
Cooking Oil Updated 6 Oct, 5:25 pm	Chicken Biryani, Chicken Burger, French Fries, Hakka Noodles, Masala Dosa, Samosa, Veg Biryani, Veg Burger	14950 ml	5000 ml	OK	Save
Dosa Batter Updated 6 Oct, 5:25 pm	Masala Dosa	9000 g	4000 g	OK	Save
Lime Updated 6 Oct, 5:25 pm	Lime Soda	60 pcs	30 pcs	OK	Save
Milk Updated 6 Oct, 5:25 pm	Cold Coffee, Masala Chai	19800 ml	8000 ml	OK	Save
Onion Updated 6 Oct, 5:25 pm	Chicken Biryani, Chicken Burger, Hakka Noodles, Paneer Wrap, Veg Biryani, Veg Burger	19950 g	5000 g	OK	Save

Figure 12: Inventory: stock and reorder levels, with ingredients below the reorder level flagged

Bill	Token	Student	Subtotal	GST	Discount	Total	Payment	Reference
B-0008 6 Oct, 5:25 pm	#0008	Aarav Sharma	₹230.00	₹11.50	—	₹241.50	UPI - paid	UPI17913075489040008
B-0005 6 Oct, 12:20 pm	#0006	Priya Iyer	₹220.00	₹11.00	—	₹231.00	Campus wallet - pending	CW-2026100612200005
B-0004 5 Oct, 1:00 pm	#0005	Aarav Sharma	₹190.00	₹9.50	—	₹199.50	UPI - paid	UPI2026100513000004
B-0003 2 Oct, 4:30 pm	#0003	Rohan Verma	₹380.00	₹19.00	- ₹20.00	₹379.00	Cash - paid	Cash
B-0002 1 Oct, 1:10 pm	#0002	Priya Iyer	₹230.00	₹11.50	- ₹10.00	₹231.50	Campus wallet - paid	CW-2026100113100002
B-0001 1 Oct, 12:45 pm	#0001	Aarav Sharma	₹200.00	₹10.00	—	₹210.00	UPI - paid	UPI2026100112451001

Figure 13: Billing: every bill with GST, discount, payment state, reference and cash collection

9.3 Insert, Update, View and Delete — Reflected in the Database

Each operation below was performed in the UI and then checked with a SELECT query in MySQL.

Insert: registering a new student

**Order ahead.
Skip the queue.**

Pick your food, pay by UPI or campus wallet, and collect it at your lane when your token shows ready.

Create a student account

Full name
Neha Kapoor

Phone (optional)
9123400007

Email
neha.kapoor@woxsen.edu.in

Password
.....

Create account

Already have an account? [Log in](#)

Figure 14: Registration form: a new student account is created

```
mysql> SELECT user_id, name, email, phone, role, LEFT(password_hash,7) AS hash_prefix FROM users WHERE email = 'neha.kapoor@woxsen.edu.in';
```

user_id	name	email	phone	role	hash_prefix
7	Neha Kapoor	neha.kapoor@woxsen.edu.in	9123400007	STUDENT	\$2b\$10\$

The new row appears in `users` with role `STUDENT`. Only the bcrypt hash (prefix `$2b$10$`) is stored, never the password.

Insert across five tables: placing an order

Aarav Sharma placed the order shown in Figures 6–7 (1 × Chicken Biryani, 1 × Samosa, 2 × Masala Chai, paid by UPI). The transaction created the following rows:

```
mysql> SELECT order_id, user_id, order_date, status, pickup_location, pickup_lane FROM orders WHERE order_id = 8;
```

order_id	user_id	order_date	status	pickup_location	pickup_lane
8	3	2026-10-06 17:25:48	CONFIRMED	Main Canteen	Lane 2

```
mysql> SELECT order_item_id, order_id, item_id, quantity, unit_price, quantity*unit_price AS line_total FROM order_items
WHERE order_id = 8;
```

order_item_id	order_id	item_id	quantity	unit_price	line_total
17	8	4	1	160.00	160.00
18	8	9	1	30.00	30.00
19	8	11	2	20.00	40.00

```
mysql> SELECT bill_id, order_id, subtotal, tax, discount, total_amount FROM bills WHERE order_id = 8;
```

bill_id	order_id	subtotal	tax	discount	total_amount
8	8	230.00	11.50	0.00	241.50

```
mysql> SELECT p.payment_id, p.bill_id, p.payment_method, p.payment_status, p.transaction_reference FROM payments p
JOIN bills b ON b.bill_id = p.bill_id WHERE b.order_id = 8;
```

payment_id	bill_id	payment_method	payment_status	transaction_reference
7	8	UPI	SUCCESS	UPI17913075489040008

The same transaction deducted ingredients according to the recipes (Chicken Biryani uses 150 g chicken and 200 g rice; Samosa 60 g potato; each Masala Chai 100 ml milk and 3 g tea leaves):

Stock before the order

ingredient_name	quantity_available	unit
Chicken	15000.000	g
Basmati Rice	25000.000	g
Potato	30000.000	g
Milk	20000.000	ml
Tea Leaves	1500.000	g

Stock after the order

ingredient_name	quantity_available	unit
Chicken	14850.000	g
Basmati Rice	24800.000	g
Potato	29940.000	g
Milk	19800.000	ml
Tea Leaves	1494.000	g

Update: moving the order through the kitchen

The screenshot shows the 'Orders' section of the Canteen management system. The interface includes a navigation bar with 'Canteen', 'Dashboard', 'Orders' (active), 'Menu', 'Inventory', and 'Billing'. The user is logged in as 'Rajesh Kumar · admin' with a 'Log out' option. Below the navigation bar, the 'Orders' section has a sub-header 'Move each order along as the kitchen works on it.' and a 'Refresh' button. A status filter bar shows 'Active' (selected), 'Pending', 'Confirmed', 'Preparing', 'Ready', 'Completed', 'Cancelled', and 'All'. The main table lists orders with columns: Token, Student, Items, Pickup, Total, Payment, and Status. The table contains four orders, with the first one (#0008) being the focus of the update.

Token	Student	Items	Pickup	Total	Payment	Status
#0008 6 Oct, 5:25 pm	Aarav Sharma	1× Chicken Biryani, 1× Samosa, 2× Masala Chai	Main Canteen Lane 2	₹241.50	UPI - paid	Preparing
#0007 6 Oct, 12:40 pm	Rohan Verma	1× Veg Burger, 1× Masala Chai	Main Canteen	₹100.00	No bill	Pending
#0006 6 Oct, 12:20 pm	Priya Iyer	2× Hakka Noodles, 1× Lime Soda	Food Court Lane 2	₹231.00	Campus wallet - pending	Preparing
#0005 5 Oct, 1:00 pm	Aarav Sharma	1× Veg Biryani, 1× Brownie	Main Canteen Lane 1	₹199.50	UPI - paid	Ready

At the bottom of the interface, a confirmation message is displayed: '#0008 → preparing'.

Figure 15: Staff moved the order to "Preparing"; the confirmation message is shown

```
mysql> SELECT order_id, status FROM orders WHERE order_id = 8;
```

order_id	status
8	PREPARING

Insert: adding a menu item

Add menu item

Name: Veg Club Sandwich

Category: Snacks

Description: Triple-layer grilled sandwich with veggies and cheese

Price (₹): 75

Prep time (min): 8

Rating (0-5, optional): 4.2

Image URL (optional):

☒ Available to order

Buttons: Cancel, Add item

Item	Category	Price	Prep time	Rating	On menu	Edit	Delete
Chicken Burger					4.4		
Crispy chicken patty with lettuce and mayo					4.5		
Paneer Wrap					4.1		
Tandoori paneer tikka rolled in a soft roti					4.7		
Veg Burger					4.0		
Aloo-veg patty with onion, tomato and sauce					4.2		
Chicken Biryani					4.6		
Hyderabadi dum chicken biryani with raita					4.0		
Hakka Noodles					4.3		
Indo-Chinese stir-fried veg noodles							
Veg Biryani							
Mixed vegetable dum biryani with raita							
Masala Dosa							
Crisp dosa with potato masala, chutney, sambar							
French Fries							
Salted crispy potato fries							
Samosa							
Two potato samosas with green chutney							
Cold Coffee	Beverages	₹60	5 min	4.4			
Chilled blended coffee with milk							
Lime Soda	Beverages	₹40	3 min	4.1			
Fresh lime with soda, sweet or salted							

Figure 16: Add menu item form filled in by the administrator

Canteen							
Dashboard Orders Menu Inventory Billing				Rajesh Kumar - admin Log out			
Chicken Biryani	Main Course	₹160	20 min	4.7		Edit	Delete
Hyderabadi dum chicken biryani with raita							
Hakka Noodles	Main Course	₹90	12 min	4.0		Edit	Delete
Indo-Chinese stir-fried veg noodles							
Veg Biryani	Main Course	₹120	18 min	4.2		Edit	Delete
Mixed vegetable dum biryani with raita							
Masala Dosa	South Indian	₹70	10 min	4.6		Edit	Delete
Crisp dosa with potato masala, chutney, sambar							
French Fries	Snacks	₹60	8 min	4.0		Edit	Delete
Salted crispy potato fries							
Samosa	Snacks	₹30	5 min	4.3		Edit	Delete
Two potato samosas with green chutney							
Veg Club Sandwich	Snacks	₹75	8 min	4.2		Edit	Delete
Triple-layer grilled sandwich with veggies and cheese							
Cold Coffee	Beverages	₹60	5 min	4.4		Edit	Delete
Chilled blended coffee with milk							
Lime Soda	Beverages	₹40	3 min	4.1		Edit	Delete
Fresh lime with soda, sweet or salted							
Masala Chai	Beverages	₹20	4 min	4.8		Edit	Delete
Hot spiced Indian tea							
Brownie	Desserts	₹70	3 min	4.5		Edit	Delete
Warm chocolate walnut brownie							

Categories			
Name	Description	Items	
Burgers & Wraps	Burgers, wraps and rolls	3	Edit Delete

Figure 17: The new item appears in the menu table (highlighted) with the “Item added” message

```
mysql> SELECT item_id, category_id, item_name, price, preparation_time, availability, rating FROM menu_items WHERE item_name = 'Veg Club Sandwich';
```

item_id	category_id	item_name	price	preparation_time	availability	rating
14	4	Veg Club Sandwich	75.00	8	1	4.2

Delete blocked by referential integrity

The screenshot shows the 'Menu' management page of the Canteen application. The page has a navigation bar with 'Canteen', 'Dashboard', 'Orders', 'Menu' (active), 'Inventory', and 'Billing'. The user is 'Rajesh Kumar - admin' and can 'Log out'. The 'Menu' section has a sub-header 'Turn items off when they run out; students see them as sold out.' and buttons for 'Add category' and 'Add item'.

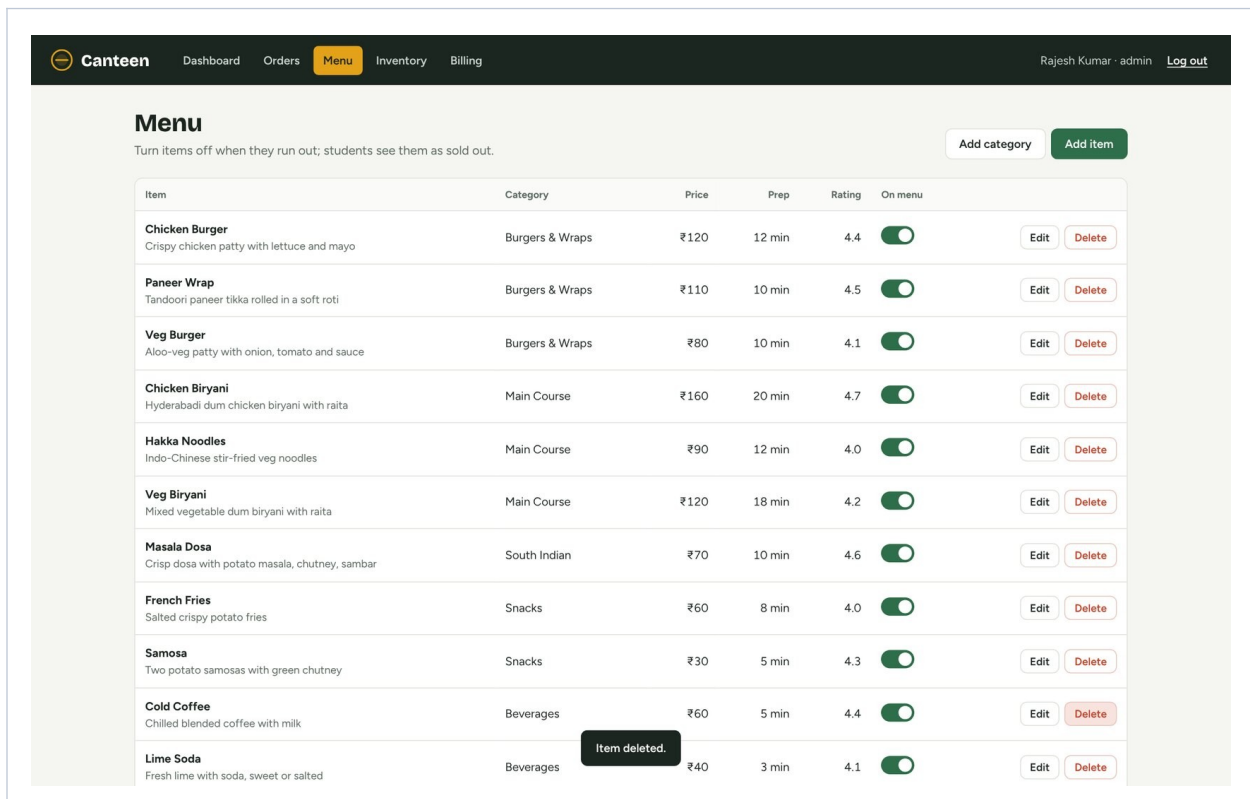
Item	Category	Price	Prep	Rating	On menu	Actions
Chicken Burger Crispy chicken patty with lettuce and mayo	Burgers & Wraps	₹120	12 min	4.4	☒	<button>Edit</button> <button>Delete</button>
Paneer Wrap Tandoori paneer tikka rolled in a soft roti	Burgers & Wraps	₹110	10 min	4.5	☒	<button>Edit</button> <button>Delete</button>
Veg Burger Aloo-veg patty with onion, tomato and sauce	Burgers & Wraps	₹80	10 min	4.1	☒	<button>Edit</button> <button>Delete</button>
Chicken Biryani Hyderabadi dum chicken biryani with raita	Main Course	₹160	20 min	4.7	☒	<button>Edit</button> <button>Delete</button>
Hakka Noodles Indo-Chinese stir-fried veg noodles	Main Course	₹90	12 min	4.0	☒	<button>Edit</button> <button>Delete</button>
Veg Biryani Mixed vegetable dum biryani with raita	Main Course	₹120	18 min	4.2	☒	<button>Edit</button> <button>Delete</button>
Masala Dosa Crisp dosa with potato masala, chutney, sambar	South Indian	₹70	10 min	4.6	☒	<button>Edit</button> <button>Delete</button>
French Fries Salted crispy potato fries	Snacks	₹60	8 min	4.0	☒	<button>Edit</button> <button>Delete</button>
Samosa Two potato samosas with green chutney	Snacks	₹30	5 min	4.3	☒	<button>Edit</button> <button>Delete</button>
Veg Club Sandwich Triple-layer grilled sandwich with veggies and cheese	Snacks	₹75	8 min	4.2	☒	<button>Edit</button> <button>Delete</button>
Cold Coffee Chilled blended coffee with milk	Beverages	₹60	5 min	4.4	☒	<button>Edit</button> <button>Delete</button>

A red error message is displayed over the 'Delete' button for the 'Chicken Biryani' item: "This item appears in past orders. Mark it unavailable instead."

Figure 18: Deleting Chicken Biryani is refused because the item appears in past orders

The application tried `DELETE FROM menu_items WHERE item_id = 4`. MySQL rejected it because `order_items.item_id` references the item with `ON DELETE RESTRICT` (error 1451), and the API turned the error into the message shown. Order history is therefore never broken; the item can be marked unavailable instead.

Delete: removing the test item



The screenshot shows the 'Menu' management interface of a canteen system. The header includes navigation links: Canteen, Dashboard, Orders, Menu (active), Inventory, and Billing. The user is logged in as 'Rajesh Kumar - admin' with a 'Log out' link. Below the header, the 'Menu' section has a subtitle 'Turn items off when they run out; students see them as sold out.' and two buttons: 'Add category' and 'Add item'.

Item	Category	Price	Prep	Rating	On menu	
Chicken Burger Crispy chicken patty with lettuce and mayo	Burgers & Wraps	₹120	12 min	4.4	☒	Edit Delete
Paneer Wrap Tandoori paneer tikka rolled in a soft roti	Burgers & Wraps	₹110	10 min	4.5	☒	Edit Delete
Veg Burger Aloo-veg patty with onion, tomato and sauce	Burgers & Wraps	₹80	10 min	4.1	☒	Edit Delete
Chicken Biryani Hyderabadi dum chicken biryani with raita	Main Course	₹160	20 min	4.7	☒	Edit Delete
Hakka Noodles Indo-Chinese stir-fried veg noodles	Main Course	₹90	12 min	4.0	☒	Edit Delete
Veg Biryani Mixed vegetable dum biryani with raita	Main Course	₹120	18 min	4.2	☒	Edit Delete
Masala Dosa Crisp dosa with potato masala, chutney, sambar	South Indian	₹70	10 min	4.6	☒	Edit Delete
French Fries Salted crispy potato fries	Snacks	₹60	8 min	4.0	☒	Edit Delete
Samosa Two potato samosas with green chutney	Snacks	₹30	5 min	4.3	☒	Edit Delete
Cold Coffee Chilled blended coffee with milk	Beverages	₹60	5 min	4.4	☒	Edit Delete
Lime Soda Fresh lime with soda, sweet or salted	Beverages	₹40	3 min	4.1	☒	Edit Delete

A black tooltip with the text 'Item deleted.' is visible over the 'Delete' button of the 'Cold Coffee' item.

Figure 19: The test item is deleted and no longer listed

```
mysql> SELECT COUNT(*) AS rows_found FROM menu_items WHERE item_name = 'Veg Club Sandwich';
```

```
+-----+
| rows_found |
+-----+
|          0 |
+-----+
```

10. Implementation Details

10.1 Technology Stack and Architecture

The system follows a three-tier architecture. The browser runs the single-page UI, the Node.js / Express server exposes a JSON REST API and serves the static files, and MySQL stores the data and enforces the integrity rules.

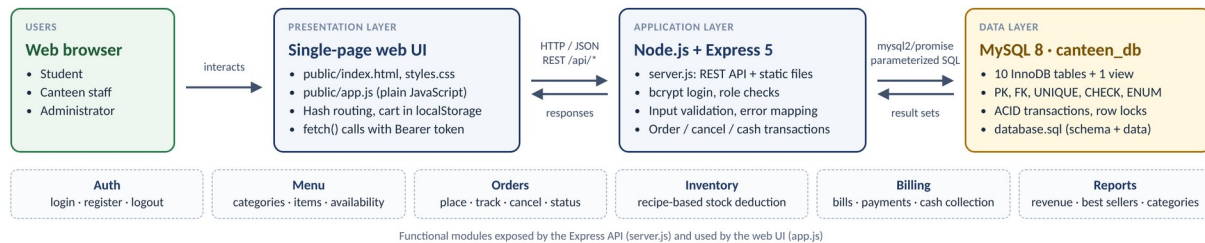


Figure 20: System architecture

Table 34: Technology stack

Layer	Technology	Role
Presentation	HTML5, CSS3, JavaScript	public/index.html, styles.css and app.js: menu, tray, orders, bill, dashboard and staff screens; hash-based routing.
Application	Node.js, Express 5	server.js: authentication, role checks, validation, transactions and reports.
Connectivity	mysql2/promise	Connection pool (10 connections) with parameterized queries.
Security	bcryptjs, random tokens	Password hashing; 48-character session tokens sent as Bearer headers.
Database	MySQL 8.0, InnoDB	10 tables and 1 view with PK, FK, UNIQUE, CHECK and ENUM constraints.
Version control	Git, GitHub	Source code, database.sql and documentation.

10.2 Database Connectivity

Connection settings are read from a `.env` file (not committed to Git) and a connection pool is created once when the server starts:

server.js — configuration and connection pool

```
// ---- tiny .env loader (no extra dependency) ----
const envFile = path.join(__dirname, '.env');
if (fs.existsSync(envFile)) {
  for (const line of fs.readFileSync(envFile, 'utf8').split(/\r?\n/)) {
    const m = line.match(/^\s*([A-Z_]+)\s*=\s*(.*)\s*$/);
    if (m && process.env[m[1]] === undefined) process.env[m[1]] = m[2];
  }
}

const pool = mysql.createPool({
  host: process.env.DB_HOST || 'localhost',
  port: Number(process.env.DB_PORT || 3306),
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD || '',
  database: process.env.DB_NAME || 'canteen_db',
  dateStrings: true,
  decimalNumbers: true,
  connectionLimit: 10,
});
```

10.3 Key Code Snippets

Login with bcrypt and parameterized SQL

```
app.post('/api/login', wrap(async (req, res) => {
  const { email, password } = req.body || {};
  if (!email || !password) throw new ApiError(400, 'Enter your email and password.');
```

```
  const [[user]] = await pool.query('SELECT * FROM users WHERE email = ?', [email.trim()]);
  if (!user || !(await bcrypt.compare(password, user.password_hash)))
    throw new ApiError(401, 'Email or password is incorrect.');
```

```
  const token = crypto.randomBytes(24).toString('hex');
  const safe = { user_id: user.user_id, name: user.name, email: user.email, role: user.role };
  sessions.set(token, safe);
  res.json({ token, user: safe });
}));
```

Role-based access control

```
function auth(...roles) {
  return (req, res, next) => {
    const token = (req.headers.authorization || '').replace('Bearer ', '');
    const user = sessions.get(token);
    if (!user) return res.status(401).json({ error: 'Please log in again.' });
    if (roles.length && !roles.includes(user.role))
      return res.status(403).json({ error: 'You do not have access to this.' });
    req.user = user;
    next();
  };
}
const STAFF = ['ADMIN', 'STAFF'];
```

Recipe-based stock adjustment (used when placing and cancelling orders)

```
// Adjust ingredient stock for an order. sign = -1 deduct, +1 restore.
async function adjustStock(conn, orderId, sign) {
  const [needs] = await conn.query(
    `SELECT inv.inventory_id, ing.ingredient_name, ing.unit, inv.quantity_available,
      SUM(mii.quantity_required * oi.quantity) AS needed
    FROM order_items oi
    JOIN menu_item_ingredients mii ON mii.item_id = oi.item_id
    JOIN ingredients ing ON ing.ingredient_id = mii.ingredient_id
    JOIN inventory inv ON inv.ingredient_id = ing.ingredient_id
    WHERE oi.order_id = ?
    GROUP BY inv.inventory_id, ing.ingredient_name, ing.unit, inv.quantity_available
    FOR UPDATE`, [orderId]);
  for (const n of needs) {
    if (sign < 0 && n.quantity_available < n.needed)
      throw new ApiError(409, `Not enough ${n.ingredient_name} in stock for this order.`);
    await conn.query(
      `UPDATE inventory SET quantity_available = quantity_available + ? WHERE inventory_id = ?`,
      [sign * n.needed, n.inventory_id]);
  }
}
```

Placing an order as one transaction

```
app.post('/api/orders', auth(), wrap(async (req, res) => {
  const { items, pickup_location, payment_method } = req.body || {};
  if (!Array.isArray(items) || !items.length) throw new ApiError(400, 'Your cart is empty.');
```

```
  if (!['UPI', 'CAMPUS_WALLET', 'CASH'].includes(payment_method)) throw new ApiError(400, 'Choose a payment method.');
```

```
  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    const ids = items.map((i) => Number(i.item_id));
```

```

const [menu] = await conn.query(
  'SELECT item_id, item_name, price, availability FROM menu_items WHERE item_id IN (?)', [ids]);
let subtotal = 0;
for (const it of items) {
  const m = menu.find((x) => x.item_id === Number(it.item_id));
  const qty = Number(it.quantity);
  if (!m) throw new ApiError(400, 'One of the items is no longer on the menu.');
```

if (!m.availability) throw new ApiError(409, `\${m.item\_name} is sold out right now.`);

if (!Number.isInteger(qty) || qty < 1 || qty > 20) throw new ApiError(400, 'Quantity must be between 1 and 20.');

it.unit_price = m.price;

subtotal += m.price * qty;

}

// simple lane assignment: lanes 1-3 rotate

const [{ n }] = await conn.query('SELECT COUNT(*) AS n FROM orders');

const lane = `Lane \${((n % 3) + 1)}`;

const [o] = await conn.query(
 'INSERT INTO orders (user_id, status, pickup_location, pickup_lane) VALUES (?, ?, ?, ?)',
 [req.user.user_id, 'PENDING', pickup_location || 'Main Canteen', lane]);

const orderId = o.insertId;

await conn.query('INSERT INTO order_items (order_id, item_id, quantity, unit_price) VALUES ?',
 [items.map((i) => [orderId, Number(i.item_id), Number(i.quantity), i.unit_price])]);

await adjustStock(conn, orderId, -1);

subtotal = money(subtotal);

const tax = money(subtotal * TAX_RATE);

const [b] = await conn.query(
 'INSERT INTO bills (order_id, subtotal, tax, discount, total_amount) VALUES (?, ?, ?, 0, ?)',
 [orderId, subtotal, tax, money(subtotal + tax)]);

// UPI / wallet are simulated as instant success; cash is paid at the counter.

const online = payment_method !== 'CASH';

const ref = online
 ? `\${payment\_method === 'UPI' ? 'UPI' : 'CW-'}\${Date.now()}\${String(orderId).padStart(4, '0')}` :
 null;

await conn.query(
 'INSERT INTO payments (bill_id, payment_method, payment_status, transaction_reference) VALUES
 (?, ?, ?, ?)',
 [b.insertId, payment_method, online ? 'SUCCESS' : 'PENDING', ref]);

if (online) await conn.query('UPDATE orders SET status = 'CONFIRMED' WHERE order_id = ?', [orderId]);

await conn.commit();

res.status(201).json({ order_id: orderId });

} catch (e) {

await conn.rollback();

throw e;

} finally {

conn.release();

}

});

The prices used for the order and the bill are read from `menu_items` inside the transaction, so a total sent by the browser is never trusted. Constraint violations raised by MySQL (error numbers 1062, 1451, 1452 and 3819) are converted into clear messages for the user.

10.4 REST API

Table 35: REST API endpoints

Method & endpoint	Access	Tables used
POST /api/login	Public	users
POST /api/register	Public	users (INSERT, role STUDENT)
POST /api/logout	Logged in	—
GET /api/categories	Public	categories, menu_items
POST PUT DELETE /api/categories	Admin	categories
GET /api/menu	Public	menu_items, categories
POST PUT DELETE /api/menu	Admin	menu_items
PATCH /api/menu/:id/availability	Staff, admin	menu_items
POST /api/orders	Logged in	menu_items, orders, order_items, inventory, bills, payments
GET /api/my/orders	Logged in	orders, order_items, bills, payments
POST /api/my/orders/:id/cancel	Order owner	orders, inventory
GET /api/orders	Staff, admin	orders, users, order_items, bills, payments
PATCH /api/orders/:id/status	Staff, admin	orders, inventory (on cancel)
GET PATCH /api/inventory	Staff, admin	inventory, ingredients, menu_item_ingredients
GET /api/bills	Staff, admin	bills, orders, users, payments
POST /api/bills/:id/collect-cash	Staff, admin	bills, payments, orders
GET /api/reports	Staff, admin	all ordering and billing tables

10.5 Order Processing Flow

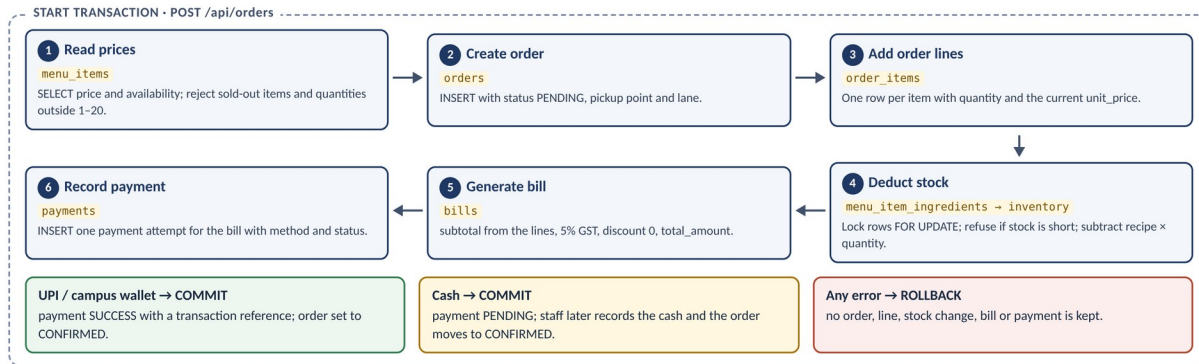


Figure 21: Order placement transaction: the six steps and how it ends

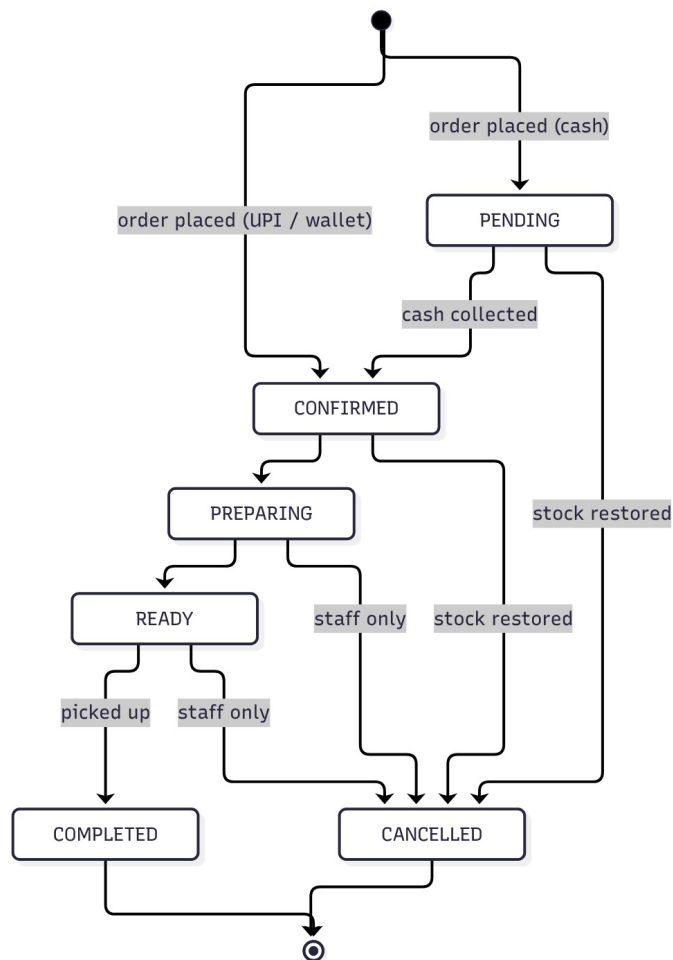


Figure 22: Order status lifecycle implemented in the application

10.6 Integrity and Security Measures

- **Integrity in the database:** named PRIMARY KEY, FOREIGN KEY, UNIQUE and CHECK constraints and ENUM domains apply even when data is changed directly in MySQL.

- **Atomic transactions:** order placement, cancellation, status changes and cash collection each run in a transaction with `SELECT ... FOR UPDATE` row locks.
- **No SQL injection:** every query uses `?` placeholders filled by the mysql2 driver.
- **Passwords:** stored only as bcrypt hashes; a CHECK constraint rejects anything shorter than a hash.
- **Authorization:** every API route declares which roles may call it; the UI also hides screens a role cannot use.
- **Trusted totals:** prices and totals are computed on the server from the database.

11. Testing

11.1 Testing Approach

Testing covered two levels. Database integrity tests ran invalid statements directly against MySQL to confirm that each constraint rejects bad data. Functional tests used the running application in a browser and verified the result in MySQL. All results below were observed on MySQL 8.0 with the sample data from database.sql.

11.2 Database Integrity Test Cases

Table 36: Database integrity test cases

ID	Test case	Expected result	Actual MySQL result	Status
TC-01	Duplicate email	UNIQUE uq_users_email rejects the row	ERROR 1062: Duplicate entry 'aarav.sharma@woxsen.edu.in' for key 'users.uq_users_email'	Pass
TC-02	Invalid phone number	CHECK chk_users_phone rejects the row	ERROR 3819: Check constraint 'chk_users_phone' is violated.	Pass
TC-03	Invalid role	ENUM domain rejects the value	ERROR 1265: Data truncated for column 'role' at row 1	Pass
TC-04	Negative menu price	CHECK chk_menu_price rejects the row	ERROR 3819: Check constraint 'chk_menu_price' is violated.	Pass
TC-05	Rating above 5	CHECK chk_menu_rating rejects the row	ERROR 3819: Check constraint 'chk_menu_rating' is violated.	Pass
TC-06	Menu item with unknown category	FK fk_menu_items_cat rejects the row	ERROR 1452: Cannot add or update a child row: foreign key constraint fails	Pass
TC-07	Delete category that still has items	ON DELETE RESTRICT blocks the delete	ERROR 1451: Cannot delete or update a parent row: foreign key constraint fails	Pass
TC-08	Delete menu item that appears in past orders	ON DELETE RESTRICT blocks the delete	ERROR 1451: Cannot delete or update a parent row: foreign key constraint fails	Pass
TC-09	Same item twice in one order	UNIQUE (order_id,item_id) rejects duplicate line	ERROR 1062: Duplicate entry '1-4' for key 'order_items.uq_order_items_order_item'	Pass
TC-10	Zero quantity	CHECK chk_order_items_qty rejects the row	ERROR 3819: Check constraint 'chk_order_items_qty' is violated.	Pass
TC-11	Second bill for the same order	UNIQUE uq_bills_order rejects the row	ERROR 1062: Duplicate entry '1' for key 'bills.uq_bills_order'	Pass
TC-12	Bill total that does not add up	CHECK chk_bills_total rejects the row	ERROR 3819: Check constraint 'chk_bills_total' is violated.	Pass
TC-13	UPI payment without a transaction reference	CHECK chk_payments_txnref rejects the row	ERROR 3819: Check constraint 'chk_payments_txnref' is violated.	Pass
TC-14	Negative stock	CHECK chk_inventory_qty rejects the update	ERROR 3819: Check constraint 'chk_inventory_qty' is violated.	Pass

Example statements used in these tests:

```
-- TC-01: Duplicate email
INSERT INTO users (name,email,phone,password_hash,role) VALUES ('Test
User','aarav.sharma@woxsen.edu.in',NULL,REPEAT('x',60),'STUDENT');
-- TC-07: Delete category that still has items
DELETE FROM categories WHERE category_id = 1;
-- TC-12: Bill total that does not add up
INSERT INTO bills (order_id,subtotal,tax,discount,total_amount) VALUES (7,100,5,0,999);
-- TC-13: UPI payment without a transaction reference
INSERT INTO payments (bill_id,payment_method,payment_status) VALUES (5,'UPI','SUCCESS');
```

11.3 Functional Test Cases

Table 37: Functional test cases

ID	Test case	Action	Observed result	Status
TC-15	Student registration	Register Neha Kapoor via the UI	Row created in users with a bcrypt hash (Figure 14)	Pass
TC-16	Role-based login	Log in as student, then as admin	Student sees Menu / My orders; admin sees staff screens	Pass
TC-17	Sold-out item	Open the menu as a student	Brownie shows “Sold out” and cannot be added (Figure 6)	Pass
TC-18	Place order (UPI)	3 items, pay by UPI	orders, order_items, bills, payments rows created; status CONFIRMED	Pass
TC-19	Stock deduction	Compare stock before and after TC-18	Each ingredient reduced by recipe × quantity	Pass
TC-20	Bill calculation	Open the bill for TC-18	₹230.00 + 5% GST ₹11.50 = ₹241.50	Pass
TC-21	Status update	Staff clicks “Start preparing”	orders.status = PREPARING	Pass
TC-22	Add menu item	Admin adds Veg Club Sandwich	Row present in menu_items	Pass
TC-23	Delete protected item	Admin deletes Chicken Biryani	Refused (FK RESTRICT); clear message shown	Pass
TC-24	Delete menu item	Admin deletes the test item	Row no longer in menu_items	Pass

All 24 test cases passed. The integrity tests confirm that invalid data is rejected by the database itself, independently of the application, and the functional tests confirm that the UI and the database stay in step.

12. Conclusion and Future Enhancements

12.1 Conclusion

The Canteen Food Ordering and Billing Management System shows how a relational database can model a real campus workflow end to end. Starting from an ER model, the design was mapped to a normalized relational schema and implemented in MySQL 8 as ten tables and a view, with the integrity rules enforced by primary keys, foreign keys, UNIQUE, CHECK and ENUM constraints. A working web application built with Node.js and Express uses the database for every screen.

Separating users, categories, menu items, ingredients, stock, orders, order lines, bills and payments removes duplication and makes the data easy to update and query. Transactions keep orders, stock and bills consistent even when something fails halfway, and the SQL queries in this report provide the summaries the canteen needs every day: order history, revenue by category, best sellers, low stock and payment breakdowns.

12.2 Future Enhancements

- Integration with a real UPI / payment gateway instead of simulated online payments.
- QR-code ordering at tables and digital receipts by email.
- Order-ready and payment notifications (SMS, email or push).
- A status-history table or trigger to audit every order status change and enforce the allowed transitions in the database.
- Coupons and discount rules managed from the admin screens.
- Persistent sessions (for example JWT or a sessions table) so logins survive a server restart.
- Kitchen display screen and support for multiple counters.
- Supplier management and automatic purchase orders for low-stock ingredients.

13. References

1. Woxsen University. *DBMS Course Project – Final Presentation (Groups of Three)*, 2026. Course handout and assessment instructions.
2. Elmasri, R. and Navathe, S. B. *Fundamentals of Database Systems*, 7th Edition, Pearson.
3. Silberschatz, A., Korth, H. F. and Sudarshan, S. *Database System Concepts*, 7th Edition, McGraw-Hill.
4. Oracle Corporation. *MySQL 8.0 Reference Manual — SQL statements, constraints, InnoDB transactions and locking*.
5. OpenJS Foundation. *Node.js Documentation*.
6. Express. *Express 5.x API Reference*.
7. mysql2. *MySQL client for Node.js — documentation (promise API, prepared statements, pooling)*.
8. bcrypt.js. *Optimized bcrypt in JavaScript — documentation*.
9. MDN Web Docs. *HTML, CSS and JavaScript references (Fetch API, Web Storage, dialog element)*.

14. Contribution of Each Member

The group has three members. Abhiuday Singh is the Team Lead. The responsibilities below match the work performed and the commit history of the project repository.

Table 38: Contribution of each member

Member	Roll number	Role	Contribution
Abhiuday Singh	25WU0101006	Team Lead	Project coordination, integration oversight, DBMS design review, task allocation, final presentation coordination and repository consolidation.
Aryaman Jain	25WU0101017	Database & SQL	Relational schema support, normalization documentation, SQL DDL / DML, query preparation and report drafting.
Harsh Jha	25WU0101046	Application & Testing	UI / application support, CRUD workflow testing, screenshot preparation and final verification support.

All three members can explain the complete project, including the ER diagram, relational schema, normalization, SQL queries and the UI-to-database flow.

Appendix A: GitHub Repository

Table 39: Repository details

Field	Details
Repository URL	https://github.com/MrAbhiudaySingh/Canteen
Language / stack	JavaScript (Node.js, Express), HTML, CSS, SQL (MySQL 8)
Database script	database.sql — creates canteen_db with all tables, the view and sample data
Configuration	.env.example — copy to .env and set the MySQL password

Repository structure

```
Canteen/
├─ server.js      REST API + static file server (Express, mysql2, bcryptjs)
├─ database.sql   schema, constraints, view and sample data for canteen_db
├─ package.json  dependencies and the "npm start" script
├─ .env.example   database connection settings template
├─ README.md     setup instructions and sample logins
└─ public/
    ├─ index.html page shell
    ├─ styles.css  styles
    └─ app.js      all screens (student + staff/admin)
```

Appendix B: How to Run the Project

1. Install **MySQL 8** and **Node.js 18 or later**.
2. Load the database (this resets it to the sample data): `mysql -u root -p < database.sql`
3. Copy `.env.example` to `.env` and set `DB_PASSWORD` to your MySQL password.
4. Install dependencies and start the server: `npm install` then `npm start`
5. Open `http://localhost:3000` in a browser.

Table 40: Sample login accounts

Role	Email	Password
Student	aarav.sharma@woxsen.edu.in	Student@123
Admin	admin@canteen.woxsen.edu.in	Admin@123
Staff	sunita.staff@canteen.woxsen.edu.in	Staff@123